Practicalities

Lecture notes.
These lecture notes are available for download from Blackboard. Together with the videos and the slides of the weekly live Question & Answer session they comprise the formal content of this part of the course unit. Suggestions for those who want to do additional optional reading is mentioned below.

These notes are by Renate Schmidt, though Joe Razavi has made some small changes for this year–and any typos were introduced by him! These notes have been considerably restructured, updated and extended from notes of Andrea Schalk from the 2014–5 delivery of the course, which in turn were based on notes of Graham Gough.

If you notice any typographical or more serious errors, I (Joe) am most grateful to receive corrections and welcome suggestions for improvement.

Additional reading
The following books on the reading list include chapters on logic, which you may wish to consult for additional explanations, more examples and exercises.

Garnier, Rowan, and Taylor, John (2002). Discrete Mathematics for New Technol-ogy, IOP. Chapter 1 and 2 on Logic and Mathematical proof. Very accessibly written.

Truss, J. K. (1999). Discrete Mathematics for Computer Scientists, Addison-Wesley. Chapter 2 discussing propositional logic, transformation to normal forms and first-order logic. More dense and advanced.

Epp, Susanna S. (2011). Discrete Mathematics with Applications, Brooks/Cole. Chapter 1 and 2 on logical statements in propositional logic and quantified logical statements (propositional and first-order logic). Very accessible. Available as online book.

Jordan, D. W. and P. Smith (2008). Mathematical Techniques: An Introduction for the Engineering, Physical, and Mathematical Sciences, Oxford University Press.

Contains only a short chapter on logic, Chapter 36, which is focussed on logic gates and switching functions; goes beyond what we cover on this topic.

Several copies of each book are available from the main library. Some of these books are available as online books; where this is the case this is indicated above.

Many other excellent textbooks exist and can be found in the library; logic and formal systems is standard material in Computer Science and Mathematics. The internet and Wikipedia is also a useful additional resource. Be aware though, there are variations in the presentation and definitions between different sources and books, and also to those we use.

Chapter 3

Formal Logic Systems

The previous chapter (Chapter 2) on Statements and Proofs concentrates on statements as they are commonly made in mathematics and also computer science, using key phrases from the English language, and how to prove such statements. Many textbooks and references in mathematics and computer science use the language of propositional logic and first-order logic for such statements. Propositional logic and first-order logic are the two most important formal logic systems in computer science. Formal logic systems are important tools used in computer science to make precise statements about computer programs and encode and reason about information of the real world.

This chapter introduces and studies formal systems and logic as general tools used to test the correctness of arguments, and to reason about true and false statements. The general aim is to understand the basics of using a formal logical system. In particular, the aim will be to discuss and understand the standard propositional logic connectives, to understand equivalence of logical formulas. We will discuss the benefits of normal forms and ways to convert logical expressions into conjunctive and disjunctive normal form. For reasoning about structured information quantifiers are needed and we will discuss the standard first-order logic connectives, including universal and existential quantification, and the role they play in making precise statements.

These tools are relevant for computer science, mathematics and many real world applications. For example, in programming languages a number of the fundamental laws that are derived in this Chapter have been implemented, giving another reason why this material is important from a computer science perspective.

3.1 Propositional logic

We first outline a comparatively simple system known as **propositional logic**, or **classical logic**. The basic entities in propositional logic and logical arguments are known as **propositions**, also referred to as **propositional formulas**. Propositions are built using **connectives**, which serve to connect propositions in order to form complex propositions.
Informally, propositions are statements that are either true or false, but not both. Examples of propositions are the following, some of which are true and others are false.

- 2 divides 4
- 225 = 152
- 2 is an odd number
- 2 + 2 = 5

- Grass is green
- It is raining
- I have my umbrella

Propositions can be connected with the connectives not, and, or, if-then and iff (if and only if) to form complex propositions, which we have already seen in the previous Chapter. Examples of complex propositions are:

- Grass is green and 2 is not an odd number.
- If it is raining then I have my umbrella.

Complex propositions are referred to as **compound propositions** and are defined to be propositions built from atomic propositions and connectives. **Atomic propositions** are propositions which can't be further broken down, such as '2 divides 4' and 'The grass is green'.

When modelling logical reasoning, we don't allow ourselves to think about the intuitive meaning of atomic propositions, and focus instead on the relationships between them expressed by the connectives.

3.1.1 Propositional formulas

In order to ensure that our propositions are interesting, and to get started, we have to allow the language of propositional logic to contain propositional variables. We therefore assume that there is a list of propositional variables, P, Q, R, ... or $P_1, P_2, \ldots$, and so on. The logical notation for the connectives not, and, or, etc are

$\neg$	not, negation
$\wedge$	and, conjunction
$\vee$	or, disjunction
$\rightarrow$	implies, if...then, implication

We will also use the symbol $\leftrightarrow$ read as "if and only if" or "bi-implication".

These are referred to as propositional connectives, or propositional logic operators.

In addition we have two constants:

$\bot$	false
$\top$	true

Which represent a proposition which is always false, and one which is always true, respectively. It is also permitted to use round or square brackets in propositional formulas.

The use of the symbols of the language of propositional logic and their intuitive meaning is the same as in mathematics (and the previous chapter), so we expect to be able to model the way we use the 'Key Phrases' of Section 2.2.1.

The formal definition of a proposition, which we prefer to call propositional formula, is as follows. We assume we have been given a set of symbols, called **propositional variables** to use to represent atomic propositions.

Definition 1: Propositional formulas

- If P is a propositional variable then P is a propositional formula (called atomic formula, or simply atom).
- The expressions $\bot$ and $\top$ are propositional formulas.
- If A is a propositional formula, then $(\neg A)$ is a propositional formula.
- If A and B are propositional formulas, then $(A \wedge B)$, $(A \vee B)$, $(A \to B)$, and are propositional formulas.

In addition we use the $(A \leftrightarrow B)$ as a shorthand notation for the propositional formula $(A \to B) \wedge (B \to A)$.

In this chapter we generally use the symbols

- P, Q, R, ... and $P_1, P_2, P_3, \ldots$ to denote propositional variables, and
- A, B, C, ... and $A_1, A_2, A_3, \ldots$ to denote propositional formulas.

Definition 1 is an example of an inductive definition. We study such definitions formally in Semester 2 when we study recursion.

Note the brackets that are part of the definition of propositional formulas. This leads to far too many brackets in formulas. The following notational convention is therefore assumed in propositional logic. In particular, we assume the following binding precedence for propositional connectives (from highest to lowest).

$$\begin{array}{cc} \neg & \\ \vee & \wedge \\ \to & \leftrightarrow \end{array}$$

This allows us to omit brackets, and write, for example,

$$P \wedge Q \to R \qquad \text{instead of } ((P \wedge Q) \to R)$$

In addition, we assume that in an expression using only $\wedge$ or using only $\vee$, these bracket from left to right, so we can write,

$$P \wedge Q \wedge R \qquad \text{instead of } ((P \wedge Q) \wedge R).$$

Note that if an expression mixes $\wedge$ and $\vee$ then we must add explicit brackets, because according to our convention they have equal binding precedence.

Omitting brackets is useful, but not essential. Brackets help make it absolutely clear what we mean. So when in doubt about the binding, feel free to use more brackets than the notational convention would force you to do.

Notice that you might find something odd about the last part of the bracketing conventions. You might feel that the order of expressions which are joined together by "and" doesn't matter. However, this is a property of the meaning of "and" which we have to *prove* holds in our model of logic. Two propositional formulas are equal if and only if once we have inserted all the omitted brackets they are identical as strings of symbols.

Definition 2: Subformula

Suppose A is propositional formula. A (propositional) subformula of A is any substring of the formula A, that is a propositional formula according to the formal definition above, taking also into consideration the convention about the use of brackets.

Note that A is also a substring of A (and a propositional formula), so A is a subformula of A. This is a somewhat informal definition; in Semester 2 we will see how to give a fully formal inductive definition of the notion of a subformula of another formula.

3.2 Propositional natural deduction

In this section we are going to discuss natural deduction as a formal proof system for performing correct reasoning and constructing correct logical arguments.

There are many ways to define deduction systems. In this course we only discuss natural deduction, but you have the opportunity to learn about other deduction approaches in years two, three and four. Compared to other deduction systems such as tableau, resolution and SAT solving, which all essentially use only proof by contradiction (and are well suited for automated reasoning), natural deduction has the advantage that it is close to actual mathematical reasoning and how mathematical proofs are typically presented.[1] Mathematical arguments are not limited to proof by contradiction, they typically include proof by case analysis, and they allow working with infinitely many formulas. An important property of mathematical proofs is that assumptions can be made at any time. Natural deduction systems are designed to cater for all this and consequently provide more flexibility than other deduction systems. A characteristic property of natural deduction systems is that all connectives are explicitly catered for in the inference rules, which is not the case in all other systems.

3.2.1 Basic concepts and the idea of natural deduction

We take a (logical) argument to be a list of assumptions (or premises) together with a conclusion which is (judged to be) supported by the assumptions. We may think of an argument as a law that asserts: from a list of assumptions, we can draw a certain conclusion.

In our propositional natural deduction system an argument is represented by a **judgement** of the general form:

$$A_1, ..., A_n \vdash A$$

where $A_1, ..., A_n$ and A are propositional formulas. We have a list of propositional formulas on the left of the turnstile symbol $\vdash$ and one propositional formula on the right. We think of such a judgement as an argument denoting that

‘A is derivable from the list of formulas $A_1, ..., A_n$‘,

or ’A is a logical consequence of the list $A_1, ..., A_n$‘, or ‘ $A_1, ..., A_n$therefore A’.

The formulas A_i on the left-hand side of the turnstile are known as the **antecedents** and the formula A on the right-hand side as the **consequent** of the given judgement.

It makes sense to think of the A_i as assumptions from which A follows. Because writing lists of propositional formulas can be a bit tedious it is customary to use this more compact notation for judgements:

$$\Gamma \vdash A$$

where Γ is a list of propositional formulas sometimes called premises of the judgement.

Not every argument (or judgement) is correct. But our interest is correct arguments, and we are interested in understanding how correct arguments can be constructed. Judgements which are correct in the technical sense we define are called **valid**.

[1] Systems used in automated reasoning will be covered in COMP21111 on Logic and Modelling, COMP24412 on Knowledge-Based AI, and COMP60332 on Automated Reasoning and Verification.

Intuitively, the following is a correct argument:

If it is raining then I have my umbrella. It is raining.
Therefore, I have my umbrella.

We can express this argument as the following judgement, if we think of P as expressing ‘It is raining‘ and Q as expressing ‘I have my umbrella‘.

$$P \to Q, P \vdash Q$$

This judgement is valid.

In contrast, $P \to Q, Q \vdash P$ is not a valid judgement. If we suppose that P expresses ‘It is raining‘ and Q expresses ‘I have my umbrella‘, intuitively the following does not make sense.

If it is raining then I have my umbrella. I have my umbrella.
Therefore, it is raining.

Note that what we are interested in here is whether the judgement is *logically* correct, not whether it expresses something true about the world. Roughly speaking, a judgement is valid if the conclusion follows from the premises considered *alone* without any supporting (or contradicting!) knowledge being used from outside sources.

For example, the argument ‘if I am a pirate’s old boot, the moon is made of green cheese. I am a pirate’s old boot. Therefore, the moon is made of green cheese’ is a valid argument—in fact when we translate it into a judgement it has exactly the same form as the previous example.
On the other hand, consider the argument:
If my shoes are wet then I am unhappy. I just stepped in a huge puddle.

Therefore, I am unhappy.

This is *not* a correct argument, because we have not written down all the knowledge about the world we are using. If we translate this into a judgement, we could use P for the proposition ‘my shoes are wet’, Q for the proposition ‘I am unhappy’ and R for the proposition ‘I just stepped in a huge puddle’. Then we can write our judgement as

$$P \to Q, R \vdash Q.$$

But now we can see that there is nothing in the argument that tells us how P and R are connected. And indeed, this judgement is not valid.

If we wanted to model this scenario logically, we would need to explicitly state the extra information. One way to do this would be to give the following argument:

If my shoes are wet then I am unhappy. I just stepped in a huge puddle.
If I have stepped in a huge puddle, then my shoes are wet.
Therefore I am unhappy.

This argument *does* state enough information to see that its conclusion follows from its premises without using common sense or experience. Using the same variable names as above, it would have the form

$$P \to Q, R, R \to P \vdash Q$$

which is valid.

3.2.2 Our natural deduction system

So far we have only formally modelled propositional formulas. All of the discussion of validity in the previous section is intuitive. Now we turn to formally modelling what the connectives mean, by giving rules which allow us to derive new judgements from given ones. These rules are closely related to the rules of proof for the 'Key Phrases' in Chapter 2.

We present a system called **natural deduction** to infer that a propositional formula A is a logical consequence of $A_1, ..., A_n$. Our natural deduction system consists of a set of inference rules for constructing a deduction (proof) showing that

$$A_1, ..., A_n \vdash A \text{ is a valid judgement.}$$

The inference rules are basically instructions for creating new judgements in a deduction. In our natural deduction system the inference rules operate on judgements because the basic elements of a natural deduction proof is a judgement. The general form of an inference rule is as follows:

$$\frac{\Gamma_1 \vdash A_1 \quad \Gamma_2 \vdash A_2 \quad ... \quad \Gamma_n \vdash A_n}{\Gamma \vdash A}$$

The judgements above the horizontal line are called the **premises** of the rule and the judgement below the horizontal line is referred to as the **derived judgement**, or the **conclusion**. We can think of an inference rule as an if-then statement in a program. The idea is that a rule is applied when there are judgements in our partially constructed deduction that match the premises of the rule. Applying the rule will then add the judgement below the horizontal line (the conclusion) to the deduction.

We only use inference rules, where either n = 0, in which case the rule has no premises and is an axiom, or n is 1, 2 or 3, in which case the rule is a one or multiple premise rule.

Our use of the terminology 'premise' is ambiguous, because we use the word both for the premise of a judgement, and the premise of a rule of inference with which correct judgements are derived, although in the former case we mostly refer to premises as assumptions. It is important to understand that a natural deduction system is a more complicated system with a more complicated language, that does not just work with propositional formulas, but with judgements, because the aim is to construct and study correct arguments involving propositional formulas.

Now we give and discuss the inference rules of our propositional natural deduction system. Each rule is named and an informal description and explanation is provided.

For each connective (except for $\leftrightarrow$) there are one or two introduction rules, and one or two elimination rules. (Recall that we consider $\leftrightarrow$ to be a shorthand.)

For conjunction there is one introduction rule and two elimination rules:

Definition: Conjunction Introduction

If A is derivable from a set of formulas, and B is derivable from the same set, then $A \wedge B$ is derivable from this set as well.

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge \text{Intro}$$

Definition: Conjunction Elimination

If $A \wedge B$ is derivable from a set of formulas, then so is A, and also B.

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} \wedge \text{Elim} \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} \wedge \text{Elim}$$

The Conjunction Introduction rule constructs a new conjunction $A \wedge B$ in the judgement $\Gamma \vdash A \wedge B$, if we know that both A and B can be inferred from the set Γ (in symbols, both $\Gamma \vdash A$ and $\Gamma \vdash B$ are known). It is called an introduction rule, because (in the conclusion) it introduces a new formula on the right-hand side of the turnstile of a judgement, in this case $A \wedge B$.

For conjunction there are two elimination rules. They have the same premise $\Gamma \vdash A \wedge B$. When applied to this judgement the left rule derives $\Gamma \vdash A$ and the right rule derives $\Gamma \vdash B$. We can think of elimination rules as breaking down the formula on the right-hand side of the turnstile of a judgement.

The next rules are the rules for disjunction.

Definition: Disjunction Introduction

If A is derivable from a set, then so is $A \vee B$, and also $B \vee A$.

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \vee \text{Intro} \qquad \frac{\Gamma \vdash A}{\Gamma \vdash B \vee A} \vee \text{Intro}$$

Definition: Disjunction Elimination (proof by cases):

If $A \vee B$ is derivable from a set and C is derivable from the set along with A, and also from the set along with B, then C is derivable from the set alone.

$$\frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C}{\Gamma \vdash C} \vee \text{Elim}$$

There are two Disjunction Introduction rules. They say that if A is derivable from a set, then so is $A \vee B$, and also $B \vee A$.

The Disjunction Elimination rule is special because it involves the discharge of the assumptions A and B in the second and third premise of the rule. The rule can be viewed as realising proof by cases common in mathematical arguments. That is, if we know $A \vee B$ holds (in the context of some assumptions Γ) and the two cases,

(i) A alone yields C and

(ii) B alone yields C,

can be shown, then C is inferrable (from Γ), and A and B can be discharged. Note that there must be three independent proofs of each of the judgements in the premise. C must be independently derivable from Γ and A, and also from Γ and B, only then does it follow that Γ yields C.

A special case is illustrated by the following example: A might mean $x \in S$ and B might mean $x \notin S$. If C is independently inferrable from both $x \in S$ and $x \notin S$ then C is inferrable outright from Γ (since $A \vee B$ is a tautology in this case and thus $\Gamma \vdash A \vee B$). Hence, C does not depend on whether $x \in S$ or $x \notin S$ and these assumptions can be discharged.

There are two inference rules for implication.

Definition: Implication Introduction:

If B is derivable from A and a set, then $A \to B$ is derivable from the set.

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \to B} \to \text{Intro}$$

Definition: Implication Elimination (modus ponens):

If A is derivable from a set, and $A \to B$ is derivable from the same set, then B is derivable from this set.

$$\frac{\Gamma \vdash A \quad \Gamma \vdash A \to B}{\Gamma \vdash B} \to \text{Elim}$$

The Implication Introduction rule says, if B is derivable from A and a set, then $A \to B$ is derivable from the set. The intuition is that when Γ together with A yield B then Γ yields $A \to B$ outright and we can discharge the formula A. This reflects how mathematicians construct proofs for implications: they assume the condition A and show that the right-hand side of the implication follows (recall the discussion in Chapter 2.)

It is important to note, when we work backwards and read the rule bottom-up, in order to derive $A \to B$, the Implication Introduction rule requires that A is added as an assumption and B is derived from A together with Γ . We can think of A as a temporary extra assumption from which B must follow.[2] If it does then we can remove it because $A \to B$ follows outright from Γ . We say A is 'discharged' during the application of the rule. This is an example of an assumption discharge rule.[3] (The Disjunction Elimination rule above is also an assumption discharge rule, because application of the rule also involves discharging assumptions, namely A and B.) The second rule, the Implication Elimination rule, says that if A is derivable from a set, and $A \to B$ is derivable from the same set, then B is derivable from this set. This rule is often referred to by the traditional Latin name 'modus ponens'.

The intuition of the Negation Introduction rule below is: If assuming a set Γ and A leads to a contradiction then A cannot hold and we can infer that $\neg A$ must hold in the presence of Γ . It can be used to construct proofs using the principle of reductio ad absurdum, i .e ., proof by contradiction.

Definition: Negation Introduction (reductio ad absurdum):

If A and a set leads to a contradiction, then $\neg A$ can be inferred from the set.

$$\frac{\Gamma, A \vdash \bot}{\Gamma \vdash \neg A} \neg \text{Intro}$$

[2] Note that there is usually an inference in the proof of $\Gamma, A \vdash B$, which activates the assumption A by using the Axiom rule below.

[3] The Implication Introduction rule is strongly related to the deduction theorem of propositional logic (proved by Herbrand in 1930). Not all logics have the deduction theorem, but if the deduction theorem holds then the logic is a nice logic. So this makes propositional logic nice.

Definition: Negation Elimination:

If A is derivable from a set, and also $\neg A$ is derivable from the set, then anything (including $\bot$) is derivable from the set.

$$\frac{\Gamma \vdash A \quad \Gamma \vdash \neg A}{\Gamma \vdash B} \neg \text{ Elim}$$

In the Negation Elimination rule the formula B in the conclusion can be any formula, since when we have a contradiction (when A and $\neg A$ are derivable) then anything follows. Inference becomes trivial once a contradiction has been found.

The two rules for double negation simply reflect that $\neg\neg A$ is equivalent to A.

Definition: Double Negation Introduction:

If A is derivable from a set, then $\neg\neg A$ is derivable from the same set.

$$\frac{\Gamma \vdash A}{\Gamma \vdash \neg\neg A} \neg\neg \text{ Intro}$$

Definition: Double negation Elimination:

If $\neg\neg A$ is derivable from a set, then A is derivable from the same set.

$$\frac{\Gamma \vdash \neg\neg A}{\Gamma \vdash A} \neg\neg \text{ Elim}$$

Any deduction also needs a starting point. The Axiom rule provides a starting point.

Definition: Axiom (starting point):

A can always be inferred from A and a set of formulas.

$$\frac{}{\Gamma, A \vdash A} \text{Ax}$$

It is a rule with no premises and is therefore always applicable. The rule can be used to assume any formula A; it says that A can always be inferred from A and a set of formulas. In particular, A may also be one of the given assumptions, and this is how it is typically used. If A is one of the given assumptions then the rule activates this assumption and moves to the right of the $\vdash$, where it can then used in the inferences.[4] A may be any propositional formula. This means the rule can introduce any formula as an assumption into a deduction.[5] We present an example shortly.

Finally we need some rules and conventions which allow us to work with the list of assumptions.

[4] Our ND system has the property that the main inferences are performed with formulas on the right of the $\vdash$. For instance, it can be broken down by one of the elimination rules or used in an inference using an introduction rule, either as the main formula or in a side premise.

[5] A better name for the Axiom rule is probably the assumption rule, because it is the main assumption making rule in natural deduction systems.

Definition: Weakening:

New assumptions may be introduced at any point in a deduction.

$$\frac{\Gamma \vdash B}{\Gamma, A \vdash B}\ \text{Weakening}$$

The second assumption making rule is the Weakening rule. It says if Γ yields B then Γ with A, for any formula A, yields B. The Weakening rule allows a formula to be added the list of assumptions on the left-hand side of a judgement at any point in the deduction. The difference to the Axiom rule is that it does not derive a new consequent on the right-hand side of $\vdash$. The Axiom rule, in contrast, introduces a formula A as a new assumption on the left-hand side and as a new consequent on the right-hand side.

We also need two crucial assumptions about our natural deduction system are the following:

(i) We assume the order of the assumptions in judgements does not matter. This means that we regard the judgement

... B, ... , B' , ... $\vdash$ A

as being the same as

... B' , ... , B, ... $\vdash$ A.

(ii) We assume that $\leftrightarrow$ has been equivalently replaced in a pre-processing step before the application of the deduction system.

In other natural deduction systems this is handled by a permutation rule. We minimise the use of so-called structural rules in our system, of which permutation is one, because these operate on occurrences of formulas and unnecessarily complicate the task of finding a proof. But, for studying proof-theoretic properties of deduction systems they are useful.

Finally, the reader might wonder about the constants $\bot$ and $\top$. The meaning of $\bot$ is explained by the Negation Introduction rule which allows us to use $\bot$ if we have proved it, and the Negation Elimination rule which is the only rule which allows us to prove $\bot$ since the formula B in that rule can be anything.

Strictly speaking, we should add an extra rule to assert that $\top$ means something which is always true. Such a rule would have the following form:

$$\frac{}{\vdash \top}\ \top\ \text{Intro}$$

However, the *only* use for this rule in our system is to be able to derive $\top$, so you will never need to use it in any of the exercises or questions in this unit. This may make $\top$ seem rather useless; its value will become clearer once we discuss logical equivalence.

We now explore how we can use these rules together to show that more complicated judgements are valid.

3.2.3 Deductions

In our natural deduction system, a deduction is a sequence of judgements, where each judgements has been derived from preceding judgements by the application of one of the inference rules. The judgement we aim to prove should always be in the last line of a deduction. In that case the deduction is a proof of this judgement.

For example, the judgement

$$P, Q \vdash P \wedge Q$$

may be proved using this formal deduction.

1. $P, Q \vdash P$	Axiom
2. $P, Q \vdash Q$	Axiom
3. $P, Q \vdash P \wedge Q$	1, 2, $\wedge$ Intro

The first and second judgements have been derived by the Axiom rule. It's obvious that the second judgement was obtained by application of the Axiom rule, but the first was, too, because we are assuming that the order of formulas to the left of the $\vdash$ does not matter (see (i) at the end of the previous section). This implies that both 1 and 2 are correct instances of inferences with the Axiom rule. The third judgement has been obtained by applying the Conjunction Introduction rule to judgements 1 and 2. As $P, Q \vdash P \wedge Q$ is the last judgement in the deduction, the sequence of judgements 1 to 3 above is a proof of $P, Q \vdash P \wedge Q$.

Overall, this tells us that from nothing at all we may infer that if P and Q hold then $P \wedge Q$ holds. This may appear terribly obvious, but once one studies longer deductions it becomes much less clear what may be derived in the system, and how one might do so.

Let us see if we can produce a proof of the judgement:

$$A \to (B \wedge C), A \vdash B$$

Lines 1 and 2 of our proof can be obtained by applying the Axiom rule:

1. $A \to (B \wedge C), A \vdash A$	Axiom
2. $A \to (B \wedge C), A \vdash A \to (B \wedge C)$	Axiom

Applying Implication Elimination (or modus ponens) to lines 1 and 2, gives line 3:

3. $A \to (B \wedge C), A \vdash B \wedge C$	1, 2, $\to$ Elim

Line 4 is now obtained by applying the first Conjunction Elimination rule to line 3

4. $A \to (B \wedge C), A \vdash B$	3, $\wedge$ Elim

Since 4 is the judgement we wanted to show, the sequence 1 to 4 is a proof of this judgement, which is what we wanted to show.

Notice how each line in the deduction has a justification on the right-hand side, which says how the judgement was obtained from previous judgements by the application of an inference rule. This is an important part of the deduction and it would not be correct without this information.

It can be quite hard to know which assumptions are needed to find a proof. So let us now look at some examples with significantly larger deductions and give some tips which may help. First, let us prove this claim:

$$A \to (B \to C) \vdash B \to (A \to C).$$

The first steps in any deduction are always assumption making steps. Often some creativity and a bit of trial and error is needed to come up with the correct assumptions for a proof of the judgement. In propositional logic, by arguing backwards the judgement that we aim to prove usually gives us clues about what assumptions to use.

For example, arguing backwards and considering the inference rules in our natural deduction system, one way to derive $A \to (B \to C) \vdash B \to (A \to C)$ in the last line of a proof, is by application of the Implication Introduction rule. This is possible if we have the judgement

(Subgoal 1) $$A \to (B \to C), B \vdash A \to C$$

and apply the Implication Introduction rule to it. This will give us $A \to (B \to C) \vdash B \to (A \to C)$. In order to derive $A \to (B \to C), B \vdash A \to C$, one possibility is by applying the Implication Introduction rule to the judgement

(Subgoal 2) $$A \to (B \to C), B, A \vdash C$$

Let us now see if the assumptions in this judgement (Subgoal 2) are enough to construct a proof.

1. $A \to (B \to C), B, A \vdash A \to (B \to C)$	Axiom
2. $A \to (B \to C), B, A \vdash A$	Axiom
3. $A \to (B \to C), B, A \vdash B \to C$	1, 2, $\to$ Elim
4. $A \to (B \to C), B, A \vdash B$	Axiom
5. $A \to (B \to C), B, A \vdash C$	3, 4, $\to$ Elim
6. $A \to (B \to C), B \vdash A \to C$	5, $\to$ Intro
7. $A \to (B \to C) \vdash B \to (A \to C)$	6, $\to$ Intro

Success! This is a natural deduction proof of the judgement we aimed to prove. Note: lines 1 to 5 are a proof of Subgoal 2, Subgoal 1 is then proved from that in line 6 and one more step gives line 7, which is the judgement we wanted to prove.

Next, we give a natural deduction proof of

$$(A \vee B) \to C \vdash (A \to C) \wedge (B \to C)$$

Again, we initially try to work backwards to attempt to obtain some clues of how to start. The desired final judgement can be derived from

(Subgoal 1) $(A \vee B) \to C \vdash A \to C$ and

(Subgoal 2) $(A \vee B) \to C \vdash B \to C$

using the Conjunction Introduction rule. In turn these judgements can be derived respectively from

(Subgoal 3) $(A \vee B) \to C, A \vdash C$ and

(Subgoal 4) $(A \vee B) \to C, B \vdash C$

using Implication Introduction. These are the first subgoals we want to prove. The assumptions in these give us promising starting points for a proof.

1. $(A \vee B) \to C, A \vdash (A \vee B) \to C$ Axiom
2. $(A \vee B) \to C, A \vdash A$ Axiom
3. $(A \vee B) \to C, A \vdash A \vee B$ 2, $\vee$ Intro
4. $(A \vee B) \to C, A \vdash C$ 1,3, $\to$ Elim

An alternative would be the Negation Elimination rule (proof by contradiction) but this rule should only be tried if no other rules help. This proves Subgoal 3. Next, let's perform

5. $(A \vee B) \to C \vdash A \to C$ 4, $\to$ Intro

This proves Subgoal 1. Let us now aim to prove Subgoal 4.

6. $(A \vee B) \to C, B \vdash (A \vee B) \to C$ Axiom
7. $(A \vee B) \to C, B \vdash B$ Axiom
8. $(A \vee B) \to C, B \vdash A \vee B$ 7, $\vee$ Intro
9. $(A \vee B) \to C, B \vdash C$ 6,8, $\to$ Elim

This proves Subgoal 4, and one more inference gives us Subgoal 2:

10. $(A \vee B) \to C \vdash B \to C$ 9, $\to$ Intro
11. $(A \vee B) \to C \vdash (A \to C) \wedge (B \to C)$ 5, 10, $\wedge$ Intro

This proves the judgement of the last line is valid, which was our aim.

Notes.

(i) There can be several different proofs of a valid judgement.

(ii) The A, B and C in the inference rules can be matched with any propositional formulas.

(iii) The same rule may be applied several times in a proof.

(iv) It is useful to think of the formulas on the left-hand side of the $\vdash$ as the assumptions that we have at our disposal, and the formulas on the right-hand side as consequences. To perform the deductions, we can only work with formulas on the right-hand side of the turnstile.

(v) In propositional logic, the judgement we want to prove tells us a lot about the assumptions to make and the rules to apply. It is often a good idea to work backwards and develop a strategy of which rules to apply to derive the judgement we want to prove, as we have done in the previous examples.

Natural deduction proofs laid out in the form of trees

We have presented natural deduction proofs in linear form, but they can also be presented in tree form. This is illustrated below, which gives tree presentations of the two natural deduction proofs above. The idea should be clear. The deductions are written as trees with applications of the Axiom rule at the leaves and the proved judgements at the root.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{A \to (B \to C), B, A \vdash B}\,\text{Axiom} \quad \dfrac{\dfrac{}{A \to (B \to C), B, A \vdash A}\,\text{Axiom} \quad \dfrac{}{A \to (B \to C), B, A \vdash A \to (B \to C)}\,\text{Axiom}}{A \to (B \to C), B, A \vdash B \to C}\,\to \text{Elim}}{A \to (B \to C), B, A \vdash C}\,\to \text{Intro}}{A \to (B \to C), B \vdash A \to C}\,\to \text{Intro}}{A \to (B \to C) \vdash B \to (A \to C)}\,\to \text{Intro}$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{(A \vee B) \to C, A \vdash A}\,\text{Axiom}}{(A \vee B) \to C, A \vdash A \vee B}\,\vee \text{Intro} \quad \dfrac{}{(A \vee B) \to C, A \vdash (A \vee B) \to C}\,\text{Axiom}}{(A \vee B) \to C, A \vdash C}\,\to \text{Elim}}{(A \vee B) \to C \vdash A \to C}\,\to \text{Intro} \quad \dfrac{\dfrac{\dfrac{\dfrac{}{(A \vee B) \to C, A \vdash B}\,\text{Axiom}}{(A \vee B) \to C, A \vdash A \vee B}\,\vee \text{Intro} \quad \dfrac{}{(A \vee B) \to C, B \vdash (A \vee B) \to C}\,\text{Axiom}}{(A \vee B) \to C, B \vdash C}\,\to \text{Elim}}{(A \vee B) \to C \vdash B \to C}\,\to \text{Intro}}{(A \vee B) \to C \vdash (A \to C) \wedge (B \to C)}\,\wedge \text{Intro}$$

Exercises on natural deduction

CExercise 1. (On Sheet 4) Give natural deduction proofs of the following judgements.

Note that you must prove exactly the judgement asked for (and not use the equivalences in the next section, for example, to replace them with formulas you prefer! They are not needed and it's the point of of this section to see that rules like those in our ND system are sufficient.)

(i) $A \wedge B \vdash B \wedge A$

(ii) $A \vdash (A \to (B \wedge C)) \to B$

(iii) $(A \vee B) \to C \vdash A \to C$

(iv) $\vdash A \leftrightarrow \neg\neg A$

Hint: First express $\leftrightarrow$ using $\to$ and $\wedge$ and construct a ND proof for $\vdash (A \to \neg\neg A) \wedge (\neg\neg A \to A)$

(v) $\neg A, \neg B \vdash \neg(A \vee B)$

(vi) $\neg(A \wedge B) \vdash \neg A \vee \neg B$

Exercise 2. This exercise is about giving partial deductions.

(i) Derive, from the judgements $\Gamma \vdash B$ and$\Gamma \vdash \neg B$ the judgement $\Gamma \vdash A$, for arbitrary propositional formulas A and B.

(ii) Derive, from $\Gamma \vdash A \vee B$ and $\Gamma \vdash \neg A$, the judgement $\Gamma \vdash B$.

(iii) Derive, from $\Gamma \vdash A \to B$ and $\Gamma \vdash A \to C$, the judgement $\Gamma \vdash A \to (B \wedge C)$.

EExercise 3. (On Sheet 4) For each connective $\star \in \{\wedge, \vee, \to\}$, consider the judgement

$$P \to Q \vdash (P \star R) \to (Q \star R)$$

and do the following:

- Choose some real-world example propositions for the atomic propositions P, Q, and R and work out whether the argument expressed by the judgement is logically correct
- For those judgements which you think express a logically correct argument, give a deduction of the judgement in our natural deduction system.

3.2.4 Tautologies and equivalent formulas

Definition 3: Tautology

A formula A is called a **tautology** if and only if the judgement

$$\vdash A$$

is valid.

This looks a bit strange because we have no list of formulas on the left-hand side of the $\vdash$, which means we have to derive A without using any assumptions! This might be puzzling at first, because the Axiom rule, which needs an assumption, is the only way we have of starting a proof. However, certain rules, such as $\to$ Intro, have extra assumptions in the premisses when compared with the conclusion. This means that reading the proof from bottom to top, they make extra assumptions appear, or when reading the proof from top to bottom, they delete an assumption which we made earlier. If we delete all the assumptions made at some point in the derivation, we will have a derivation whose last judgement has an empty list of assumptions, showing that its conclusion is a tautology.

CExercise 4. (On Sheet 4) Give natural deduction proofs to show the following are tautologies.

(i) $A \to (A \wedge (A \vee B))$

(ii) $(A \vee (A \wedge B)) \to A$

(iii) $A \to (\neg A \to B)$

(iv) $A \vee \neg A$

(v) $\neg(A \wedge B) \vee A$

Definition 4: Equivalent formulas

Let A and B be formulas. Then we say that A and B are **equivalent** and write $A \equiv B$ if and only if we can derive both judgements

$$A \vdash B$$

and

$$B \vdash A$$

There are important examples of logical equivalences which we call "fundamental logical equivalences", several of which have traditional names. It is important to note that some of these names are a bit misleading, because $\equiv$ is absolutely *not* equality! For example some of the equivalences below are called "commutative", but the formulas on either side of the equivalence are not identical, and hence not equal to each other. The fundamental equivalences are as follows.

Theorem 3.1: Fundamental logical equivalences

For all formulas A and B, we have:

$\neg\neg A \equiv A$ double negation / involution

$A \wedge A \equiv A$ idempotence

$A \vee A \equiv A$ idempotence

$A \wedge B \equiv B \wedge A$ commutative

$A \vee B \equiv B \vee A$ commutative

$A \wedge (B \wedge R) \equiv (A \wedge B) \wedge R \equiv A \wedge B \wedge R$ associative

$A \vee (B \vee R) \equiv (A \vee B) \vee R \equiv A \vee B \vee R$ associative

$A \wedge (B \vee R) \equiv (A \wedge B) \vee (A \wedge R)$ distributive

$A \vee (B \wedge R) \equiv (A \vee B) \wedge (A \vee R)$ distributive

$A \wedge (A \vee B) \equiv A$ absorption

$A \vee (A \wedge B) \equiv A$ absorption

$\neg(A \wedge B) \equiv \neg A \vee \neg B$ De Morgan‘s law

$\neg(A \vee B) \equiv \neg A \wedge \neg B$ De Morgan‘s law

$A \to B \equiv \neg A \vee B$

$A \to B \equiv \neg(A \wedge \neg B)$

$A \to B \equiv \neg B \to \neg A$ contrapositive

$A \leftrightarrow B \equiv (A \to B) \wedge (B \to A)$

$A \leftrightarrow B \equiv (\neg A \vee B) \wedge (A \vee \neg B)$

$A \leftrightarrow B \equiv (A \wedge B) \vee (\neg A \wedge \neg B)$

These equivalences give us an insight into another way to reason about propositional formulas. In particular, we would like to use them to simplify complicated formulas and reduce the number of different ways of expressing the same information we have to think about. However, to do this we would need to prove lots of properties of logical equivalence. Recalling that $\equiv$ is *not* equality, because propositional formulas are just strings, we see that we should not assume that $\equiv$ has the usual properties of equality.

In particular, we don’t know that we can replace subformulas inside a large formula with equivalence subformulas and claim that the result is equivalent to the large subformula we started with! The natural deduction system makes it awkward to prove properties like this, so

in the next sections we will introduce an alternative way to give meaning to the connectives.

The following core exercise gives you a chance to prove some of the fundamental equivalences for yourself, and the extensional exercise demonstrates what reasoning about equivalence is like if we stick to using the natural deduction system.

CExercise 5. (On Sheet 4) Prove the following fundamental equivalences:

(i) $A \wedge A \equiv A$ idempotence

(ii) $A \vee A \equiv A$ idempotence

(iii) $A \wedge (A \vee B) \equiv A$ absorption

(iv) $A \vee (A \wedge B) \equiv A$ absorption

EExercise 6. (On Sheet 4) Suppose that for formulas A and B, we can derive the judgement $A \vdash B$. Prove that the judgement $C \to A \vdash C \to B$ is derivable for all formulas C.

If $A \equiv B$, what does your result allow you to conclude about the formulas $C \to A$ and $C \to B$? Prove that your answer is correct.

Remarks

Very many different variations of natural deduction systems exist. So don't get confused if you refer to textbooks and the natural deduction system looks different. We have chosen a natural deduction system that includes several elements of other variations of natural deduction systems and related sequent systems, while at the same time being intuitive and practical.

While conventionally natural deduction derivations are given in tree form, we have presented them in linear form, because actual human reasoning is linear! The construction of linear proofs is easier, and proofs tend to be shorter although the system is not optimal in this respect (the system is not as general as it could be in order to produce shortest proofs). As a consequence the system is easier to learn and apply. There is however no loss of generality because any proofs in linear form can also be presented in tree form, as we have noted.

Tips

You might find the following tips useful for finding natural deduction proofs.

To construct a natural deduction proof for $A_1, ..., A_n \vdash A$ it is useful to:

- Work backwards and develop a strategy of how to derive A from the assumption formulas and identify the inference rules to be applied, as well as subgoals.
- It is normally a good idea to have at least all the given assumption formulas $A_1, ..., A_n$ on the left-hand sides of the judgements in a deduction.
- Activate the needed assumption formulas by using the Axiom rule:

$$\frac{}{A_1, ..., A_n \vdash A_i}\ \text{Axiom}$$

- Attempt to derive judgements so that you prove the subgoals and perform the inferences that you have identified in your strategy.
- Make new assumptions as needed for performing inferences with discharge rules ($\to$ Intro, $\neg$ Intro, $\vee$ Elim), by either using the Axiom rule or the Weakening rule.

3.2.5 Boolean semantics

All that is defined above is purely syntactic, that is, we are manipulating symbols. While the rules of Natural Deduction give meaning to the connectives, this is in terms of how we may use them to construct derivations, and this makes reasoning about some properties of our logical system awkward. It also doesn't connect logic to the outside world and explain its relevance to other paths of maths or computer science.

What is needed is a formal definition of the **semantics** of propositional formulas. Semantics gives formal meaning to formulas in terms of less syntactic mathematical objects. Semantics must be defined without ambiguity, and in such a way that the validity of arguments can be formally checked and verified.

The most simple and most widely used semantics of propositional formulas is **Boolean semantics**, which we describe in this section and then define formally in the next section. Boolean semantics is based on thinking of a formula which can be true or false in various circumstances.

A propositional formula by itself is not true or false. It becomes so only after the propositional variables have been replaced with concrete propositions, or with truth values. We therefore have to consider instantiations of the propositional variables and the effect of the connectives on truth values.

In the Boolean semantics of propositional logic there are only two truth values:

$$1 \qquad \text{and} \qquad 0$$

You may think of 1 as denoting 'true', and 0 as denoting 'false'[6]. The effect of the connectives on truth values is summarised by the following standard truth tables.

P	$\neg P$
0	1
1	0

P	Q	$P \wedge Q$
0	0	0
0	1	0
1	0	0
1	1	1

P	Q	$P \vee Q$
0	0	0
0	1	1
1	0	1
1	1	1

P	Q	$P \to Q$
0	0	1
0	1	1
1	0	0
1	1	1

We can also give a truth table for $\leftrightarrow$ based on those for $\wedge$ and $\to$ as follows.

P	Q	$P \leftrightarrow Q$
0	0	1
0	1	0
1	0	0
1	1	1

Note that we always interpret $\top$ as 1 and $\bot$ as 0.

[6] You sometimes find people using different names for the elements of this set, for example $\{T, F\}$ or $\{t, f\}$ or $\{tt, ff\}$ (COMP11212 on Fundamentals of Computation). You should have no difficulty translating between these.

The standard truth tables correspond to the natural reading of the connectives. It can be seen that:

$\neg P$ is true	iff	P is false,
$P \wedge Q$ is true	iff	both P and Q are true
$P \vee Q$ is true	iff	at least one of P or Q are true
$P \leftrightarrow Q$ is true	iff	either P and Q are both true or P and Q are both false
$P \rightarrow Q$ is **false**	iff	either $P$is true and Q is false.

Certainly from a common-sense point of view these make sense: If I have two true statements A and B then the statement we obtain by connecting these with ‘and‘ is true, but if one of them is false then the combined statement should be false. You may want to go back and read the text for the key phrases from Section 2.2 to compare what is said there with the above definitions for these operations.

Remark.

The interpretation of $\rightarrow$ can be a bit puzzling, for example, if propositions are involved that are false in the real world. Consider

If the moon is made of cheese then 2 + 2 = 5.

According to the defined semantics of $\rightarrow$ this propositional formula is true. In contrast, we have no problems with accepting the definition of the semantics of implication for statements such as the following:

If 4 is an even number then 4 is a multiple of 2.

If 4 is an even number then 4 is a prime number.

where the condition is true and the consequent is true, and the condition is false and the consequent is true. In the first example both the condition and the consequent are false (because non-sensical) and the result is true. If we were to let the result be false in this example, and also the result be false, if one (of the condition or the consequent of the implication) is false and the other one is true, then the interpretation of $\rightarrow$ would be the same as $\wedge$. This would not be intuitive, because too strong. Interpreting “If it is raining then I have my umbrella” as “It is raining and I have my umbrella” is not intuitive.

The way the interpretation of $\rightarrow$ is defined is the best there is; it works. What is important is that $P \rightarrow Q$ is true when P is true and Q is true, and $P \rightarrow Q$ is false when P is true and Q is false. To avoid puzzlement just remember that $P \rightarrow Q$ is only 0 when P is 1 and Q is 0 (or think of$P \rightarrow Q$ as an abbreviation for$\neg P \vee Q$). (End of remark)

Now, how do we interpret a complex propositional formula such as the following?

$$(\star) \qquad ((P \vee Q) \wedge \neg P) \vee \neg Q$$

As indicated above, we have to work relative to valuations for the propositional variables. (We also call these valuations ‘interpretations‘ or ‘truth assignments‘ in the Boolean semantics.) The formula contains two variables P and Q. In this example we can take as a valuation v a function $\{P, Q\} \rightarrow \{0, 1\}$ that maps the propositional variables P and Q to 1 and 0. Our interpretation is relative to a valuation, and to really understand the given propositional formula

we have to consider all possible valuations of the list of (at least the) propositional variables occurring in the formula.

In the Boolean semantics, the interpretation of a propositional formula is given by the truth table of the formula.[7] For example, the truth table for the formula ($\star$) above

is:

P	Q	$((P \vee Q) \wedge \neg P) \vee \neg Q$
0	0	1
0	1	1
1	0	1
1	1	0

We obtain this truth table by filling in a longer version of the table that helps make the calculations faster.[8]

P	Q	$P \vee Q$	$(P \vee Q) \wedge \neg P$	$((P \vee Q) \wedge \neg P) \vee \neg Q$
0	0	0	0	1
0	1	1	1	1
1	0	1	0	1
1	1	1	0	0

You may wish to also include columns for the interpretations of the subformulas $\neg P$ and $\neg Q$ in order to help avoid mistakes. The last column if called the truth value column of the formula $((P \vee Q) \wedge \neg P) \vee \neg$Q.

CExercise 7. (On Sheet 5) Construct the truth tables for the following propositional formulas, and show the truth tables of each of the pairs of formulas are the same (i.e., the truth value columns for the given formulas are the same).

(i) $P \vee Q$ and $\neg P \to Q$

(ii) $P \wedge Q$ and $\neg(P \to \neg Q)$

(iii) $P \leftrightarrow Q$ and $(P \to Q) \wedge (Q \to P)$

(iv) $\neg(P \leftrightarrow Q)$ and $(P \vee Q) \wedge (\neg P \vee \neg Q)$

(v) $P \wedge (P \vee Q)$ and $P \vee (P \wedge Q)$

(vi) $P \wedge (Q \vee R)$ and $(P \wedge Q) \vee (P \wedge R)(vii)P \vee (Q \wedge R)$ and$(P \vee Q) \wedge (P \vee R)$

EExercise 8. (On Sheet 5) Write down a propositional formula with the propositional variables P , Q, R for which the truth value column in its truth table is:

(i) 0 0 0 1 0 0 0 0

(ii) 0 1 0 1 0 0 0 1

(iii) 0 0 0 0 0 0 0 0

Explain your answers.

[7] Compare these with the rules for logical operators on page 128 of Java: Just in Time, and with those in the first handout for COMP12111—they all talk about the same operations, only that implication is omitted there.

[8] This is the same idea as the truth tables in Chapter 8 of Java: Just in Time.

3.3 More on semantics

A formal, general definition of semantics

There are many ways of defining the semantics of propositional formulas, which are relevant to different subareas of computer science and mathematics. We start of by describing the general setting for defining each of the different semantics, and how Boolean semantics fits into it. Subsequently, we give two more examples of semantics of propositional formulas relevant to computer science.

Because our propositional formulas are built using propositional variables we have to worry about how to interpret propositional variables. We cannot make any assumptions about the meaning of these variables, and so we employ a trick that is also found in the formal semantics of programming languages: Intrinsically, a propositional variable has no fixed meaning. The way we assign meaning to a propositional formula is to consider all the possible meanings a variable could have, one at a time. This leads to the concept of a valuation (i.e., an interpretation function).

Definition 5: Valuation

Assume S is the set of values that propositional formulas can take. Further, assume $P_1, P_2, ..., P_n$ is a finite list of variables. A **valuation** v for these variables is a function

$$v : \{P_1, P_2, ..., P_n\} \to S$$

that assigns a value from S to each propositional variable in the list.

We also have to assign interpretations in S to the connectives. This is done in a formal way which depends on their semantics. The interpretation of a formula is then defined as follows:

Definition 6

Suppose A is any propositional formula that contains the propositional variables $P_1, P_2, ..., P_n$, and suppose v is a valuation for these propositional variables.

The **interpretation** of A relative to valuation v is the value of the formula obtained by

- replacing each P_i by vP_i, and
- evaluating the interpretation of each connective.

If we look at all possible valuations, that is, at all the functions with the source and target given above, then we look at all the possible meanings the propositional formula could have in our semantics. This is why we assign a truth value to each propositional formula relative to a valuation. We may state properties of this semantics by quantifying over all possible valuations. This process is described in more detail for each of the semantics we discuss below. (Some computer scientists and mathematicians refer to these different semantics as models.)

3.3.1 Semantics 1: Boolean semantics

In the Boolean semantics, the values that propositional formulas can take are

$$S = \{1, 0\}$$

. S is known as the **Boolean** truth value set (or two element Boolean algebra) and is denoted $\mathbb{B}$. This means

- A Boolean valuation v is a function

  $v : \{P_1, P_2, ..., P_n\} \to \{1, 0\}$

  that assigns 1 or 0 to the list $P_1, P_2, ..., P_n$ of propositional variables.

- The Boolean interpretation of a formula A relative to v is the value of the formula obtained by replacing each P_i by vP_i and calculating the truth value of A in accordance with the standard truth tables for the connectives. Note that $\top$ is always replaced by 1 and $\bot$ is always replaced by 0.

The Boolean interpretation of a propositional formula is also called its truth value.

Interpretation of a propositional formula

Given the list P, Q of propositional variables, assume we have a valuation v that takes its values in $\{0, 1\}$. The interpretation of our running example $(\star)$ from the previous section, i.e.,

$$(\star) \qquad ((P \vee Q) \wedge \neg P) \vee \neg Q,$$

relative to v is then given by the truth value of

$$((vP \vee vQ) \wedge \neg vP) \vee \neg vQ.$$

Assume, for example, that we have a valuation v which maps P to 0 and Q to 1 . Then the value of $((P \vee Q) \wedge \neg P) \vee \neg Q$ relative to this v is

$$\begin{aligned}
&((vP \vee vQ) \wedge \neg vP) \vee \neg vQ \\
=&((0 \vee 1) \wedge \neg 0) \vee \neg 1 && \text{def } v \\
=&((0 \vee 1) \wedge 1) \vee 0 && \text{def } \neg \\
=&(1 \wedge 1) \vee 0 && \text{def } \vee \\
=&1 \vee 0 && \text{def } \wedge \\
=&1 && \text{def } \vee
\end{aligned}$$

therefore the interpretation of the formula $(\star)$ relative to the given valuation v is 1.

The interpretation of the formula $(\star)$ relative to the given v can also be read off from the truth table of $(\star)$—namely, in the third row.

P	Q	$((P \vee Q) \wedge \neg P) \vee \neg Q$
0	0	1
0	1	1
1	0	1
1	1	0

This illustrates:

- Each line in a truth table gives the interpretation of a formula for the given truth values of its propositional variables.

  It is also immediate that:

- Truth tables display all possible interpretations of a formula.

Computing the truth value (interpretation) of a formula for a given valuation is a model checking task. In software engineering, for safety critical applications, model checking is used to verify the correctness of software against their high-level specifications. Model checking is also used to verify the correctness of security protocols and other kinds of protocols.

Tautology, satisfiability, contradiction

We can now define the concept of a tautology, which we previously defined using the Natural Deduction system, using the Boolean semantics. In order to justify giving this concept the same name, we ought to prove that the definitions agree in all cases. This does work, but we postpone discussion of why until the end of the section. The Boolean semantics also allows us to define simply some notions which would have been very awkward using the Natural Deduction system.

Definition 7: Tautology - semantic definition

A propositional formula A is called a **tautology**, if the interpretation of A is 1, for all possible valuations v of the propositional variables occurring in A.

It follows from the definition of a tautology that truth tables can be used to show that a formula is a tautology. For example, the formula $(P \wedge Q) \rightarrow (Q \vee \neg P)$ is a tautology, because the entries in its truth column in the truth table for the formula are all 1.

P	Q	$(P \wedge Q) \rightarrow (Q \vee \neg P)$
0	0	1
0	1	1
1	0	1
1	1	1

By looking at its truth table above we can say that the formula$((P \vee Q) \wedge \neg P) \vee \neg Q$ is not a tautology, because not every entry in the final column is a 1.

Definition 8

A propositional formula A is **satisfiable**, if the interpretation of A is 1, for some valuation v. Otherwise, it is a **contradiction** (we also say it is unsatisfiable in this case).

For example, the formulas $(P \wedge Q) \rightarrow (Q \vee \neg P)$ and $((P \vee Q) \wedge \neg P) \vee \neg Q$ are satisfiable. We can conclude this by looking at their truth tables. In both cases there is at least one entry in their respective truth columns, which is 1.

Because the size of a truth tables is exponential in the number of variables occurring in a formula, it can be more efficient to calculate the interpretation of a formula for different valuations until one is found which returns the truth value 1 without constructing the complete truth table.

3.3.2 Semantics 2: Power set semantics

Reducing the interpretation of every propositional formula to just one value, 1 or 0, is somewhat restrictive, and not very subtle.

In the power set semantics, the values that propositional formulas can take are subsets of X, where X is an arbitrary (but fixed) non-empty set. That is, valuations are defined over $S = \mathcal{P}X$, the power set of X. In particular:

Definition 9: Power set semantics

In the **power set semantics**:

- A valuation v is a function

  $v : \{P_1, P_2, ..., P_n\} \to \mathcal{P}X$

  that assigns subsets of X to the list $P_1, P_2, ..., P_n$ of propositional variables.

- To obtain the power set interpretation of a formula A relative to v

for conjunction	$\wedge$	use intersection	$\cap$
for disjunction	$\vee$	use union	$\cup$
for negation	$\neg B$	use set complement	$X \setminus SB$
for an implication	$B \to B'$	use	$(X \setminus SB) \cup SB'$,

  where SB is the interpretation of formula B relative to v and SB' is the interpretation of formula B' relative to v.

  The interpreatation of $\bot$ is the empty set, and the interpretation of $\top$ is the whole set X.

  The set $S = \mathcal{P}X$ with the set operations $\cap, \cup$ and set complement is called the power set algebra.

In the power set semantics, the interpretation of a propositional formula A relative to a given valuation v over a set X can be calculated as follows.

- In the formula A, replace every occurrence of P_i by vP_i.
- Replace the connectives according to the instructions given in the second bullet point in the Definition above.
- Finally, calculate the value of the resulting expression in P X. This will return a subset of X.

We go back to the example $(\star)$ from earlier, namely $((P \vee Q) \wedge \neg P) \vee \neg Q$. Assume $X = \{a, b, c\}$ and assume the valuation v maps

P to $\{a, b\}$

and

Q to $\{b, c\}$.

Then the value of $((P \vee Q) \wedge \neg P) \vee \neg Q$ relative to v is

$$\begin{aligned}
&((vP \cup vQ) \cap (X \setminus vP)) \cup (X \setminus vQ) \\
=&((\{a,b\} \cup \{b,c\}) \cap (X \setminus \{a,b\})) \cup (X \setminus \{b,c\}) && \text{def } v \\
=&(\{a,b,c\} \cap \{c\}) \cup \{a\} && \text{def } \cup, \\
=&\{c\} \cup \{a\} && \text{def } \cap \\
=&\{a,c\} && \text{def } \cup
\end{aligned}$$

In the power set semantics, the notions tautology, satisfiability and contradiction are defined as follows.

Definition 10

A propositional formula A is a **tautology**, if the interpretation of A is X, for all possible valuation functions v and any non-empty set X.

A formula A is **satisfiable**, if the interpretation of A is a non-empty set, for some valuation function v and some non-empty set X. Otherwise, A is a **contradiction**.

The requirement for a tautology is quite strong. A tautology is a propositional formula which is interpreted as X, for *every* possible valuation and any non-empty set X. The requirement is less strong for satisfiability. Satisfiability just requires the existence of a valuation such that the interpretation of the propositional formula in question is non-empty, for *some* non-empty set X.

Hence in the interpretation given we can see that $((P \vee Q) \wedge \neg P) \vee \neg Q$ is satisfiable, and we can immediately conclude it is neither a tautology nor a contradiction (because its valuation is not X nor is it the empty set). We can make this conclusion after making calculations for just one valuation. This is in contrast to the Boolean semantics where we have to find at least one valuation for which the formula evaluates to 0, and one valuation where it evaluates to 1.

On the other hand in the power set semantics, there are significantly more valuations available, so checking all of them explicitly will take considerably more time. The approach of explicitly computing the interpretations is completely unfeasible for checking whether a formula is a tautology or a contradiction, as there are uncountably many non-empty sets that we can take as X and therefore also uncountably many interpretations of a formula![9]

However, we can use set-theoretic arguments and general properties of sets and operations of sets. For example, to show that

$$P \vee \neg P$$

is a tautology we can argue as follows. Let X be an arbitrary non-empty set and suppose the valuation of P is $vP = SP$, where S_P is a subset of X. In order to show that the above formula is a tautology we show that its interpretation is X, i.e.,$vP \cup (X \setminus vP) = S_P \cup (X \setminus S_P) = X$.

We can do this by showing for any $x \in X$

$$x \in S_P \cup (X \setminus S_P)$$

iff

$$x \in X$$

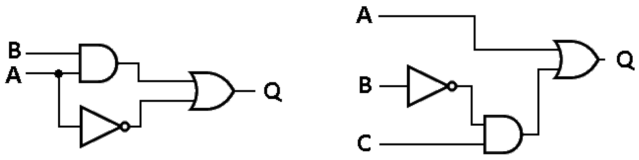
The proof of this is left as an exercise.

[9] At this stage, think of 'uncountably many' as meaning 'infinitely many'. We say a set is uncountable if it is infinite in size and there is no systematic way of listing the elements of the set.

3.3.3 Semantics 3: Logic Gates

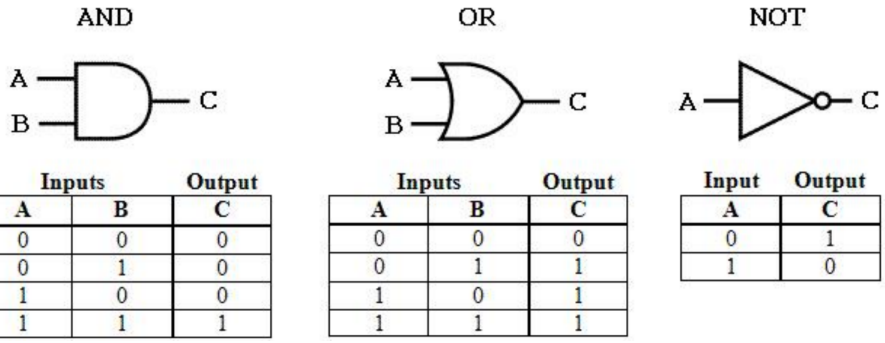
We may build yet another semantics for our formal system, using logic gates.

Logic gates can be used to build electronic switching circuits such as these:



Source: http://www.nandland.com/articles/boolean-algebra-using-look-up-tables-lut.html

These circuits are built by connecting three types of logic gates: AND gates, OR gates and NOT gates. The effects of logical gates are summarised by these tables, where 1 represents a high voltage, and 0 represents a low voltage.



Inputs		Output
A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

Inputs		Output
A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

Input	Output
A	C
0	1
1	0

Source: http://newstudent.groups.et.byu.net/Labs/Logic%20Gates/LogicGates.html

In the logic circuit semantics, each propositional variable for a given propositional formula becomes an input wire for our circuit. We use the following to connect these wires according to the connectives in the propositional formula:

for conjunction $\wedge$ use	an AND gate.
for disjunction $\vee$ use	an OR gate.
for negation $\neg$ use	an NOT gate.
for implication $\rightarrow$ use	an OR gate whose first input is negated.

The resulting circuit has one output wire. A valuation means deciding, for each input wire what voltage to put onto that input wire. Given a valuation we get two possibilities for the output wire: Either it carries a low voltage, or it carries a high voltage.

The circuit interpretation of a propositional formula is the resulting circuit. The notion of tautology is then adjusted to this semantics by demanding that for every valuation (that is, for every way of putting a voltage, onto the input wires) the output wire carries high voltage. Satisfiability means that there is at least one way of assigning voltages to the input wires such that the output wire carries high voltage.

Applications

Instead of thinking of a model as saying something about the formal system, one could think of the formal system making statements about the intended semantics. The fact that there are so many semantics (models) available is one of the reasons why the language of logic is applicable in so many ways.

Those of you taking COMP12111 have met logic as a description of what happens in an electronic circuit. Propositional logic can be used to test if two circuits are equivalent.

The conditional statements in Java use logic connectives, just with a different syntax so that they may be typed on a standard keyboard. You find more about these below.

We have seen there is an obvious way of taking our 'key phrases' and turning them into logical connectives, and so we may think of a formal system like ours as talking about definitions and proofs from mathematics. Additionally, via the power set semantics we can use our formal system to reason about of sets (of any kind) and properties. The power set semantics can be seen to provide the foundation for ontology-based knowledge processing technologies which will be covered in year four (in COMP24011 Introduction to AI, COMP24412 Knowledge-based AI and COMP60411 on Modelling Data on the Web).

But statements built from logical connectives are used in many other areas as well, for example in database queries, or to describe the result such a query ought to have.

The key phrases, or the corresponding logical connectives, allow us to make precise statements about a great variety of different topics. Because they appear in so many places it is important to understand these connectives and their intended meanings, as well as their properties.

In general, whenever people wish to formally establish something such as the correctness of a circuit or a program relative to a given specification they translate the specification into a formal system, and construct a formal semantics (model) of the circuit or program within that system, and then try to establish a formal proof using a mechanism supplied by the system. In the next section we talk about equivalent formulas and describe methods of testing if two formulas are equivalent. These methods can then also used to test equivalence of the corresponding logical circuits.

Exercises

Exercise 9. If you have a propositional formula with n propositional variables, how many possible valuations over these variables are there?

CExercise 10. (On Sheet 5) Consider the following propositional formula

$$(P \vee Q) \to \neg(P \wedge Q)$$

. (i) Calculate the Boolean interpretation of the formula relative to the valuation which maps P to 0 and Q to 1.

(ii) Calculate the power set interpretation in $X = \{a, b, c\}$ of the formula relative to the valuation which maps P to $\{a, b, c\}$ and Q to $\{c\}$.

CExercise 11. (On Sheet 5) Determine which of the following propositional formulas are tautologies? Which are contradictions? Which are satisfiable? Use truth tables to explain your answers.

(i) $P \to (Q \to P)$

(ii) $P \to (\neg P \vee Q)$

(iii) $((P \to Q) \to Q) \to Q$

(iv) $((P \to Q) \to P) \to P$

(v) $(P \to Q) \vee (Q \to R)$

(vi) $(((P \vee Q) \wedge (P \to R)) \wedge (Q \to R)) \to R$

Exercise 12. Let A and B be any propositional formulas. Show the following in the power set semantics.
(i)$A \to (A \vee B)$ is a tautology.
(ii) $(A \wedge B) \to A$ is a tautology.
(iii) If A is a tautology then $\neg$ A is a contradiction.

EExercise 13. (On Sheet 5) Let A and B be any propositional formulas. Use the Boolean semantics to show the following:
(i) Show that if A is a tautology and $A \to B$ is a tautology then B is a tautology.
(ii) Disprove (by giving a counter-example) that if $A \to B$ is a tautology and B is a tautology then A is a tautology.

Exercise 14. Give an argument that the power set semantics subsumes the Boolean semantics. Hint: Consider the power set for a one-element set.

Exercise 15. Assume that every valuation may assign only the values $\emptyset$ or X to a propositional variable. What does that mean for this model?

3.4 Equivalent formulas

As for the notion of tautology, we can give a semantic definition of equivalence. As before we use the same symbol, leaving it to later to discuss why the two different definitions define the same concept.

3.4.1 Semantic equivalence

Definition 11

We say two formulas A and B are **semantically equivalent**, and write $A \equiv B$, iff their interpretation is the same, for any valuation v (of the propositional variables occurring in A and B).

In particular, in the Boolean semantics (which we use most often because it is the simplest), two formulas A and B are semantically equivalent if they have the same truth tables. [10]
Examples of semantically equivalent formulas are the following.

double negation / involution $P \equiv \neg\neg P$

P	$\neg P$	$\neg\neg P$
0	1	0
1	0	1

idempotence: $P \equiv P \wedge P$

P	$P \wedge P$
0	0
1	1

idempotence $P \equiv P \vee P$

P	$P \vee P$
0	0
1	1

Notice that when we presented these fundamental equivalences in the chapter on natural deduction, we did so for an apritrary formula A, but here since P is a fixed propositional variable, the truth tables above prove the semantic equivalence only for the one *specific* formula P. We couldn't use a truth table to prove that the equivalence holds for all formulas A because a formula can have any number of variables, and so we would not know how big our truth table needed to be! We return to this issue below.
Here is an example of a more complex fundamental equivalence:

P	Q	$\neg P$	$\neg P \vee Q$	$P \to Q$
0	0	1	1	1
0	1	1	1	1
1	0	0	0	0
1	1	0	1	1

[10] The truth tables should include all propositional variables appearing in either A or B.

Since the fourth and the last column are the same we know that $P \to Q$ and $\neg P \vee Q$ have the same interpretation for every possible combination of input values. As a consequence, no matter which value is assigned to P and Q in our interpretation for the given valuation, the two propositional formulas are assigned the same truth value (and are therefore interpreted in the same way for every interpretation of the propositional variables). Hence, $P \to Q$ and $\neg P \vee Q$ are equivalent.

So when we are merely interested in the interpretation of propositional formulas in our semantics, we may remove the connective $\to$ from consideration entirely, and instead use negation and conjunction.[11]

In exactly the same way we can show the equivalence of two propositional formulas, by constructing their truth tables and checking that the formulas have the same truth values for every possible valuation. This shows how we can use truth tables to show that two formulas are equivalent.

The exercises above already gave several examples of propositional formulas with the same truth value columns. These are all semantically equivalent.

The following is a useful list of fundamental semantic equivalences, including some we have already discussed. In the previous section on Natural Deduction, a large number of rquivalences are given in Theorem 3.1.

It is important to note that $A \equiv B$ is not the same as$A \leftrightarrow B$, though the two notions are related. Semantic equivalence is a property of formulas, and a meta-logical notion. The symbol $\equiv$ is not a logical connective; it is notation in the meta-language, whereas $\leftrightarrow$is a symbol of the language of propositional logic (the object language).

The connection between semantic equivalence and the double implication connective is:

Theorem 3.2

$A \equiv B$ iff $A \leftrightarrow B$ is a tautology.

Also, be careful not to confuse $\equiv$ with equality! Our symbol for equality is $=$. We have $P \vee Q \equiv P \vee Q \equiv Q \vee P$ and $P \vee Q = P \vee Q, but P \vee Q \neq Q \vee P$. Two 'things' are only equal when they are the same.

3.4.2 Substitution and equivalent replacement

Substitution for propositional variables

Substitution provides a formal mechanism to replace a propositional variable within a formula by another formula.

Let A be the propositional formula

$$P \vee \neg(Q \to P)$$

.

If we want to substitute the formula $B = P \wedge Q$ for$P$ in A then we need to put B in place of every occurrence of P in A. We get $B \vee \neg(Q \to B)$, or $(P \wedge Q) \vee \neg(Q \to (P \wedge Q))$.

In the following theorem we use the following notation. Let A and $C_1, ..., C_n$ denote propositional formulas. Writing $A(P_1, ..., P_n)$ means that the propositional variables $P_1, ..., P_n$ possibly occur in the formula A. The notation $A(C_1, ..., C_n)$ then refers to the formula that is obtained, when for each i with $1 \leq i \leq n$, the formula C_i has been substituted for every occurrence of P_i in A.

[11] But note that if we are interested in a logical system for other reasons (such as the use of formulas for convenient, compact representation of information or the intuitive presentation of logical arguments) then we should not just remove connectives.

Theorem 3.3: Substitution Theorem

Let A and B be any propositional formulas. If $A(P_1, ..., P_n) \equiv B(P_1, ..., P_n)$ and $C_1, ..., C_n$ are propositional formulas, then $A(C_1, ..., C_n) \equiv B(C_1, ..., C_n)$.

Proof. Let v be a valuation for the variables occurring in $A(C_1, ..., C_n)$, $B(C_1, ..., C_n)$. We can construct a new valuation v' for the variables $P_1, ..., P_n$ such that for each i with $1 \leq i \leq n$, we set $v'P_i$ to be the interpretation of C_i relative to the original valuation v. The valuation v' has the property that the interpretation of $A(P_1, ..., P_n)$ relative to v' is the same as the interpretation of $A(C_1, ..., C_n)$ relative to the original valuation v, and the interpretation of $B(P_1, ..., P_n)$ relative to v' is the same as the interpretation of $B(C_1, ..., C_n)$ relative to the original valuation v.

Now since $A(P_1, ..., P_n) \equiv B(P_1, ..., P_n)$, the interpretation of these relative to the valuation v' is the same, and hence the interpretations of $A(C_1, ..., C_n)$ and $B(C_1, ..., C_n)$ relative to the original valuation v are the same.

Since this happens for every valuation v, we can conclude that $A(C_1, ..., C_n) \equiv B(C_1, ..., C_n)$

Note that in the proof we rely on a key property of substitutions with respect to valuations which we do not prove; to do that we would need a formal definition of substitution which would take us far beyond the scope of this course unit.

For example of the use of this theorem, from the law of double negation $P \equiv \neg\neg P$ and using the Substitution Theorem 3.3, we can conclude that

$$A \equiv \neg\neg A$$

for any propositional formula A. This in turn implies, for example, $(P \to Q) \equiv \neg\neg(P \to Q)$.

Similarly, from De Morgan‘s law $\neg(P \wedge Q) \equiv (\neg P \vee \neg Q)$, we can conclude

$$\neg(A \wedge B) \equiv (\neg A \vee \neg B)$$

for any propositional formulas A and B using the Substitution Theorem For example,
$\neg((P \vee \neg R) \wedge (P \to S)) \equiv (\neg(P \vee \neg R) \vee \neg(P \to S))$.

In a similar way it follows from the Substitution Theorem that proving versions of the fundamental equivalences involving single variables allows us to conclude the general version which we could prove using the Natural Deduction system.

Equivalent replacement

Besides substitution, equivalent replacement is an important operation to equivalently transform formulas. Substitution means putting one formula for all occurrences of a particular propositional variable (possibly repeatedly). Replacement means putting one formula for one occurrence of a subformula in another formula. It is proving this property where our semantics has an advantage over Natural Deduction.

In the following theorem we use the slightly informal notation $C(...A...)$ to refer to the formula C and stipulate that A is a subformula of C.

Theorem 3.4: Equivalent Replacement Theorem

Let A and B be propositional formulas. Suppose $A \equiv B$ and A is a subformula of a propositional formula $C(...A...)$. Then $C(...A...) \equiv C(...B...)$, where $C(...B...)$ is the result of replacing a certain occurrence of A in $C(...A...)$ by B.

We know$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$. By the Equivalent Replacement Theorem we get

$$\neg(P \wedge Q) \vee R \equiv (\neg P \vee \neg Q) \vee R$$

.

We have replaced $\neg(P \wedge Q)$ in the left-hand side by $\neg P \vee \neg Q$ to obtain the right-hand side. Also

$$\begin{aligned}
&\neg(P \wedge Q) \vee \neg Q \\
\equiv&(\neg P \vee \neg Q) \vee \neg Q && \text{by equiv. replacement and De Morgan} \\
\equiv&\neg P \vee (\neg Q \vee \neg Q) && \text{by equiv. repl. and general associativity of } \vee \\
\equiv&\neg P \vee \neg Q && \text{by equiv. repl. and } \neg Q \vee \neg Q \equiv \neg Q
\end{aligned}$$

This shows $\neg(P \wedge Q) \vee \neg Q \equiv \neg P \vee \neg Q$ without using truth tables or Natural Deduction![12]

Starting with the given formula $\neg(P \wedge Q) \vee \neg Q$ we have replaced $\neg(P \wedge Q)$ with $\neg P \vee \neg Q$ to get $(\neg P \vee \neg Q) \vee \neg Q$, by using De Morgan‘s law which says $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ and the Equivalent Replacement Theorem. In the next step we have used the equivalence called (somewhat misleadingly as noted above) associativity of $\vee$ to replace $(\neg P \vee \neg Q) \vee \neg Q$ by $\neg P \vee (\neg Q \vee \neg Q)$. And so on.

The example illustrates that we can use the fundamental laws and the Equivalent replacement theorem[13] to prove the equivalence of two formulas, namely, by equivalently transforming one into the other. In each step we have performed equivalent replacement using a fundamental equivalence. The idea of equivalent replacement is to try and find a subformula that matches the left-hand side of a fundamental equivalence and replace it by the right-hand side of the equivalence. We may also use fundamental equivalences from right-to-left.

The formula $\neg P \vee \neg Q$ is significantly simpler than the formula $\neg(P \wedge Q) \vee \neg Q$ we started with, which illustrates that the fundamental laws and the Equivalent replacement theorem[14] provide a useful basis for simplifying propositional formulas, i.e., finding simpler looking equivalent formulas.

Note that many computer languages have propositional formulas: every language that has some kind of conditional ‘if ... then‘ instructions uses them. This means that we may use the techniques learned in this section to rewrite the condition in our if statement into a more easily understandable, equivalent form. This makes it much easier to check whether one has captured the various intended cases in one‘s if statement.

3.4.3 Equivalences involving $\top$ and $\bot$

We have not yet exhausted the possibilities for simplifying formulas. In particular, equivalences involving the constants $\top$ and $\bot$ are extremely useful for this.

[12] This follows since $\equiv$ can be shown to be an equivalence relation, i.e., a reflexive, symmetric and transitive relation. We discuss these properties of relations in Semester 2.

[13] and implicitly also the substitutivity theorem when we use generalised fundamental laws

[14] and implicitly also the substitutivity theorem

Theorem 3.5: Fundamental semantic equivalences involving $\top$ and $\bot$

$A \vee \neg A \equiv \top$ excluded middle
$A \wedge \neg A \equiv \bot$ contradiction
$A \to A \equiv \top$
$A \leftrightarrow A \equiv \top$
$\neg\top \equiv \bot$
$\neg\bot \equiv \top$
$A \vee \top \equiv \top \equiv \top \vee A$
$A \wedge \top \equiv A \equiv \top \wedge A$
$A \vee \bot \equiv A \equiv \bot \vee A$
$A \wedge \bot \equiv \bot \equiv \bot \wedge A$
$A \to \bot \equiv \neg A$
$A \leftrightarrow \top \equiv A$
$\bot \to A \equiv \top$
A $\leftrightarrow \bot \equiv \neg A$

These are especially useful for simplifying formulas.

In addition, we can use equivalences involving the constants to investigate whether formulas are tautologies or contraditions.

Theorem 3.6

(i) A is a tautology iff $A \equiv \top$.

(ii) A is a contradiction iff $A \equiv \bot$.

Remark.

There are other logic systems, e.g., intuitionistic logic or first-order logic, which are not truth functional, where interpretations are defined differently, because the semantics of formulas cannot be defined using truth tables. Intuitionistic logic is also a propositional system but not all of the fundamental equivalences hold; the law of excluded middle does not hold for intuitionistic logic. First-order logic involves quantifiers over variables which takes us beyond propositional logic. We discuss first-order logic in a subsequent section (End of remark)

3.4.4 Semantic equivalence in the other semantics

Recall the definition of semantic equivalence of formulas. Let A and B denote two propositional formulas. We defined $A \equiv B$ iff the interpretation of A and B is the same, for any valuation v for the list of propositional variables occurring in A and B.

In the power set semantics, an interpretation is defined relative to the valuation v but also a non-empty set X and the interpretation of any formula is a subset of X.

This means, in the power set semantics $A \equiv B$ iff the sets interpreting A and B are the same for any valuation and any non-empty set X.

Thus, to show $A \equiv B$, we have to show that for any non-empty set X and any valuation v (over X) for the propositional variables in A and B, the interpretation of A is the same as the interpretation of B, i.e., $vA = vB$. Suppose S_A is the interpretation of A and S_B the interpretation of B relative to v. We have to show $S_A = S_B$.

In the definition of the power set semantics, we have already used the equivalence $A \to B \equiv \neg A \vee B$, namely in the definition of the interpretation of $\to$. Let the interpretation of A relative

to valuation v be S_A, and that of B be S_B (both subsets of X). Then the interpretation of $A \to B$ relative to v is $(X \setminus SA) \cup SB$, which is exactly the interpretation relative to v of $\neg A \vee B$.

When the direct proof of $S_A = S_B$ is not that easy to see, for showing $A \equiv B$, it may be useful to remember that $S_A = S_B$ can be shown by showing, for any $x \in X$: $x \in SA$ iff $x \in SB$.

The way the definition of semantic equivalence is instantiated in the logic circuit semantics is as follows. In the logic circuit semantics, two formulas A and B are equivalent iff the voltage output of the circuits representing either formula is the same for the same tuple of voltages for the input variables.

To conclude this section, we note that to test semantic equivalence it suffices to do this in one semantics; we don‘t have the test equivalence in all three semantics! The same is the case for the other tasks that we discussed (testing whether a formula is a tautology, a contradiction, whether it is satisfiable, etc). In general, it suffices to work just in one semantics. However, the flexibility to work in any one of the three semantics is useful, because it allows us to routinely transfer properties of propositional formulas to elementary set theory (via the power set semantics) or logical switching circuits (via the logic gates semantics), but also vice versa. It also allows using methods for propositional logic for elementary set theory or logical switching circuits, and vice versa. Of the three semantics, the Boolean semantics is most widely used, because it is the simplest.

Exercises

Exercise 16. In Chapter 2 we encountered the notions of associativity and commutativity of binary operations. We can adopt these notions, but instead of asking certain expressions to evaluate to the same value we ask that they are semantically equivalent.

(i) Give an argument that $\wedge$ is commutative with respect to $\equiv$, that is, $A \wedge B \equiv B \wedge A$. Now do the same for $\vee$. Hint: Consider the argument we made for $P \to Q \equiv \neg P \vee Q$ on page 20.

(ii) Give an argument that $\wedge$ and $\vee$ are associative with respect to$\equiv$.

CExercise 17. (On Sheet 5) The following tasks explore how the fundamental equivalences can be put to use now that we have the Equivalent Replacement theorem.

(i) By using the fundamental equivalences and the Equivalent Replacement theorem, show that the formula

$$(P \wedge \neg Q) \to (\neg(Q \vee R) \vee R)$$

is equivalent to

$$(\neg P \vee Q \vee \neg Q \vee R) \wedge (\neg P \vee Q \vee \neg R \vee R)$$

(ii) Simplify the formula as much as possible using the equivalences, including those for $\top$ and $\bot$, and state whether it is satisfiable, a tautology, or a contradiction.

Exercise 18. Show $A \wedge (A \vee B) \equiv A$ (one of the two absorption laws) in the power set semantics. Make sure you look up the definition of the power set semantics and read section 3.4.4.

Exercise 19. Show the idempotency laws (see Theorem 3.4) hold in the power set semantics, by showing the interpretations of $A \vee A$ and $A \wedge A$ are equal to the interpretation of A, all relative to an arbitrary given valuation.

Exercise 20. Show the following in the power set semantics.
(i) The law of double negation $A \equiv \neg\neg$A.
(ii) $A \to B$ is a tautology iff $SA \subseteq SB$, where SA and SB are respectively the interpretation of A and B relative to any valuation and any non-empty set X.
(iii) Exploit (ii) to show: $A \to (A \vee B)$ is a tautology.
(iv) Exploit (ii) to show: $(A \wedge B) \to A$ is a tautology.

Exercise 21. Show the following in the Boolean semantics.
$A \wedge \top \equiv A \equiv \top \wedge A$
$A \vee \bot \equiv A \equiv \bot \vee A$

3.5 Normal forms and Boolean functions

Propositional formulas can be written in many equivalent ways. For example,$A \to B$ is semantically equivalent to each of these formulas.

$\neg A \vee B$,
$\neg(A \wedge \neg B)$,
$\neg\neg(A \to B)$

There are in fact infinitely many ways of equivalent expressing $A \to B$. In order to avoid having to consider so many equivalent forms a key technique in logic and automated reasoning is to get rid of them by working with normal forms. A normal form is a restricted, simpler but equivalent form of a formula. The idea is to transform formulas into normal forms is a general one, also applicable to formulas and structures in mathematics and problems in many areas of computer science. (Normal forms are not always unique.)

Transforming formulas into normal form has several advantages. It allows for the simplification of formulas, thereby making the formulas easier to understand and work with.

It can be very useful to focus on a small subset of 'essential' logical concepts, eliminating for example connectives that are 'superfluous' because they can be expressed using other connectives. This makes it easier to study a formal logic system and understand its properties. Also, in the development of automated reasoning tools it is very beneficial to be able concentrate on devising efficient data structures and algorithms for a small set of concepts. This is is easier, because there is less to consider, debugging is easier and so is maintenance of the automated reasoning tool. Normalisation and working with normal forms is also a key principle in the many inference systems such as resolution and tableau systems, which are discussed in year two in "Introduction to AI", "Logic and Modelling" and "Knowledge-based AI".

Equivalence preserving normalisation gives us an alternative method for testing whether two formulas are equivalent. Rather than using the truth table method to test whether $A \leftrightarrow B$, we can transform both formulas to conjunctive normal form, which we discuss below, and test if $CNF(A) \equiv CNF(B)$. This is often easier.

The most important and widely used normal forms in computer science are conjunctive and disjunctive normal forms. We can also transform both formulas to DNF and test if DNF(A) $\equiv$ DNF (B). Writing CNF (A) is strictly an abuse of notation (and strictly speaking wrong) since the conjunctive normal form of a formula is not necessarily unique. In discussing these we will make extensive use of our notational convention which allows us to write a sequence of conjunctions (or a sequence of disjunctions) without brackets. To do this we may need to apply the so-called associativity equivalence many times. As we want to observe when we do this we shall call this simplification "flattening".

3.5.1 Conjunctive normal form

Definition 12

A propositional formula is in conjunctive normal form, if it is a conjunction of the form
$A_1 \wedge A_2 \wedge ... \wedge A_m$,
where each A_i has the form
$B_1 \vee B_2 \vee ... \vee B_n$,
and where each B_j is either $\top$, $\bot$, a propositional variable or the negation of a propositional variable and $m \geq 1$, $n \geq 1$.

For example, $(P \vee \neg R) \wedge (Q \vee \neg R \vee \neg P) \wedge (Q \vee \neg R)$ is in conjunctive normal form.

For every propositional formula we can find a semantically equivalent formula in conjunctive normal form, and there is an algorithm for doing this.

Step 1: If the formula contains any $\rightarrow$ or $\leftrightarrow$ connectives, replace them all by using the laws

$$A \rightarrow B \equiv \neg A \vee B$$
$$A \leftrightarrow B \equiv (\neg A \vee B) \wedge (A \vee \neg B)$$

Step 2: Push $\neg$ as far as possible inwards in the formula by using these variations of the De Morgan's laws.

$$\neg(A_1 \wedge A_2 \wedge ... \wedge A_n) \equiv \neg A_1 \vee \neg A_2 \vee ... \vee \neg A_n$$
$$\neg(A_1 \vee A_2 \vee ... \vee A_n) \equiv \neg A_1 \wedge \neg A_2 \wedge ... \wedge \neg A_n$$

Step 3: Eliminate all sequences of negations by using the law of double negation.

$$\neg\neg A \equiv A$$

Step 4: Eliminate all conjunctions inside disjunctions by equivalent replacement of

$$\begin{array}{l} (\ A_{(1,1)} \wedge ... \wedge A_{(1,k_1)}\ ) \\ \vee \\ (\ A_{(2,1)} \wedge ... \wedge A_{(2,k_2)}\ ) \\ \vee \\ \vdots \\ \vee \\ (\ A_{(n,1)} \wedge ... \wedge A_{(n,k_n)}\ ) \end{array}$$

with

$$\bigwedge \left\{ (B_1 \vee ... \vee B_n) \mid B_1 \in \{A_{(1,1)}, ..., A_{(1,k_1)}\}, ..., B_n \in \{A_{(n,1)}, ..., A_{(n,k_n)}\} \right\}$$

Where for any set $S = \{C_1, ..., C_m\}$ of propositional formulas, we write

$$\bigwedge S$$

for the formula $C_1 \wedge ... \wedge C_m$. If $m = 1$ we mean just C_1, and if $m = 0$ we mean $\top$.

Step 4 is the most complex step. What happens in Step 4 is that a disjunction of conjunctions is turned into a conjunction. In this conjunction each conjunct is a disjunction constructed from exactly one A_i of each of the conjunctions of the given formula. Step 4 essentially performs a form of "multiplying out".

Note we always try to apply the equivalences in Steps 1 to 3 from left to right, that is, we try to replace a formula that looks like the LHS by the RHS of the equivalence.

It is important that the steps of the algorithm are applied persistently, i.e., repeatedly and exhaustively, until the formula does not change anymore when any step is tried. The algorithm suggests a particular order in which the steps are applied, but, in fact, the steps can be applied in any order. For example, sometimes it is better to eliminate double negations (Step 3) before doing Step 2.

Suppose B is the formula returned by applying this algorithm to a propositional formula A. B is semantically equivalent to A because each step performed in the algorithm preserves

semantic equivalence. This is clear for Steps 1, 2, and 3, since they are each based on applying logical laws we have discussed in the previous section.

Step 4 also preserves semantic equivalence because the transformation can be mimicked with several steps using the general distributivity equivalences.

$$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$$

and

$$(B \wedge C) \vee A \equiv (B \vee A) \wedge (C \vee A)$$

. We have not explicitly discussed the second distributivity equivalence. However, note it follows from the first distributivity equivalence, the so-called commutativity equivalence of $\vee$, i.e., $A \vee B \equiv B \vee A$, and using equivalent replacement (several times).

We have to also convince ourselves that the algorithm always terminates. Intuitively speaking it is clear that Steps 1, 2 and 3 can only be applied finitely often.

The most complicated and expensive step is Step 4. We can show that the number of conjunctions in the formula returned is a finite number. (It is an exercise to work this out.)

Thus, the algorithm is guaranteed to terminate, returning an output B (say) which is equivalent to the input formula A. We have argued that $A \equiv B$; it remains to argue that B is in conjunctive normal form. We leave you to think about that as an exercise.

Theorem 3.7

(i) Each step of the CNF algorithm preserves semantic equivalence.

(ii) Applying the algorithm to any propositional formula

(a) terminates after finitely many steps, and

(b) results in an equivalent formula in conjunctive normal form.

Let us consider an example of converting a propositional formula into conjunctive normal form.

$$\begin{array}{lll}
(\dagger) & (P \wedge Q) \vee \neg(R \wedge (P \vee R)) & \\
& \equiv (P \wedge Q) \vee (\neg R \vee \neg(P \vee R)) & \text{Step 2, De Morgan} \\
& \equiv (P \wedge Q) \vee (\neg R \vee (\neg P \wedge \neg R)) & \text{Step 2, De Morgan} \\
& \equiv (P \wedge Q) \vee \neg R \vee (\neg P \wedge \neg R) & \text{associativity } \vee \\
& \equiv (P \vee \neg R \vee \neg P) \wedge (P \vee \neg R \vee \neg R) & \text{Step 4} \\
& \quad \wedge (Q \vee \neg R \vee \neg P) \wedge (Q \vee \neg R \vee \neg R) & \text{This is a conjunctive normal form.}
\end{array}$$

Note, however, that we can apply further simplifications here. We know that

- $P \vee \neg P \equiv \top$ and so also $P \vee R \vee \neg P \equiv \top$,
- $\neg R \vee \neg R \equiv \neg R$ and so also $P \vee \neg R \vee \neg R \equiv P \vee \neg R$ and $Q \vee \neg R \vee \neg R \equiv Q \vee \neg R$.

As a consequence we get that

$$\begin{array}{llll}
(P \vee \neg R \vee \neg P) & \wedge (P \vee \neg R \vee \neg R) & \wedge (Q \vee \neg R \vee \neg P) & \wedge (Q \vee \neg R \vee \neg R) \\
\equiv \top & \wedge (P \vee \neg R) & \wedge (Q \vee \neg R \vee \neg P) & \wedge (Q \vee \neg R) \\
\equiv & (P \vee \neg R) & & \wedge (Q \vee \neg R).
\end{array}$$

For the last step we used $A \wedge (A \vee B) \equiv A$, one of the absorption laws.

Having the propositional formula (†) in the equivalent form $(P \vee \neg R) \wedge (Q \vee \neg R)$ makes it much easier to understand what it is actually saying! It is also easier to work out whether the formula is a tautology, a contradiction or neither.

In general, formulas in conjunctive normal form obtained by the algorithm can often be significantly further simplified (as was the case in the example) by using

- the fundamental equivalences involving $\top$ and $\bot$,
- idempotence, 'associativity' and 'commutativity' of $\wedge$ and $\vee$, and the absorption laws.

3.5.2 Disjunctive normal form

One can swap the role of conjunction and disjunction and obtain a different normal form.

Definition 13

A propositional formula is in disjunctive normal form, if it is either
$\top, \bot$, or a disjunction of the form
$A_1 \vee A_2 \vee \ldots \vee A_m$,
where each A_i is of the form
$B_1 \wedge B_2 \wedge \ldots \wedge B_n$,
and where each B_j is either $\top$,$\bot$, a propositional variable or the negation of a propositional variable and $m \geq 1, n \geq 1$.

Disjunctive normal forms have the same properties as conjunctive normal forms; only the role of $\wedge$ and $\vee$ are reversed. In particular, any propositional formula A is semantically equivalent to a formula in disjunctive normal form, and there is an algorithm for finding such a formula. The algorithm is very similar to the one for CNF—we merely have to exchange the role of $\wedge$ and $\vee$.
Step 1: If the formula contains any $\rightarrow$ or $\leftrightarrow$ connectives, replace them all by using the laws

$$A \rightarrow B \equiv \neg A \vee B$$
$$A \leftrightarrow B \equiv (\neg A \vee B) \wedge (A \vee \neg B)$$

Step 2: Push $\neg$ as far as possible inwards in the formula by using these variations of the De Morgan's laws.

$$\neg(A_1 \wedge A_2 \wedge \ldots \wedge A_n) \equiv \neg A_1 \vee \neg A_2 \vee \ldots \vee \neg A_n$$
$$\neg(A_1 \vee A_2 \vee \ldots \vee A_n) \equiv \neg A_1 \wedge \neg A_2 \wedge \ldots \wedge \neg A_n$$

Step 3: Eliminate all sequences of negations by using the law of double negation.

$$\neg\neg A \equiv A$$

Step 4: Eliminate all disjunctions inside conjunctions by equivalent replacement of

$$
\begin{array}{l}
(\ A_{(1,1)} \vee ... \vee A_{(1,k_1)}\ ) \\
\wedge \\
(\ A_{(2,1)} \vee ... \vee A_{(2,k_2)}\ ) \\
\wedge \\
\vdots \\
\wedge \\
(\ A_{(n,1)} \vee ... \vee A_{(n,k_n)}\ )
\end{array}
$$

with

$$\bigvee \{(B_1 \wedge ... \wedge B_n) \mid B_1 \in \{A_{(1,1)}, ..., A_{(1,k_1)}\}, ..., B_n \in \{A_{(n,1)}, ..., A_{(n,k_n)}\}\}$$

Where for any set $S = \{C_1, ..., C_m\}$ of propositional formulas, we write

$$\bigvee S$$

for the formula $C_1 \vee ... \vee C_m$. If $m = 1$ we mean just C_1, and if $m = 0$ we mean $\bot$.

Steps 1, 2 and 3 are the same as in the CNF algorithm. The only difference is in Step 4, where we push conjunctions inwards over disjunctions.
The distributivity laws used in Step 4 are
$A \wedge (B \vee C) \equiv (A \wedge B) \vee (A \wedge C)$ and
$(B \vee C) \wedge A \equiv (B \wedge A) \vee (C \wedge A)$.
By using further simplifying transformations, as in the case of conjunctive normal form, formulas can often be reduced significantly further.

We consider the same propositional formula (†) as before as an example. Since the first three steps of the DNF algorithm are the same as in the CNF algorithm we know we can obtain

$(P \wedge Q) \vee \neg(R \wedge (P \vee R))$
$\equiv (P \wedge Q) \vee \neg R \vee (\neg P \wedge \neg R)$

in three steps as before.

This formula is in disjunctive normal form. It is no coincidence that we are done sooner here. It is often the case that obtaining one of the normal forms requires more calculations than the other, but in general we cannot know for which normal form the calculations will be shorter.

Note that again we may simplify this propositional formula further by using one of the absorption laws and commutativity of $\wedge$, which implies $\neg R \vee (\neg P \wedge \neg R) \equiv \neg R$.
Hence we may simplify the above disjunctive normal form to get the following.

$(P \wedge Q) \vee \neg(R \wedge (P \vee R))$ (3.4)
$\equiv (P \wedge Q) \vee \neg R$

The example illustrates again that generally, the obtained simplified disjunctive normal form is much more intelligible than the input formula. In this case, from looking at the the simplified DNF we can conclude that the original formula is true when either

- both P and Q are true (no matter whether R is true or false), or
- R is false (and the truth values of P and Q do not matter).

This example illustrates a general property: From the DNF of a formula, it is easy to read off interpretations that make the formula true. A similar (dual) property holds for conjunctive normal forms, where the role of $\wedge$ and $\vee$ reversed, and the role of true and false are reversed. Namely: It is easy to read off falsifying interpretations from the CNF of a formula.

In the previous section we determined for our running example:
$(P \wedge Q) \vee \neg(R \wedge (P \vee R)) \equiv (P \vee \neg R) \wedge (Q \vee \neg R)$
Looking at the CNF on the right-hand side we can immediately[15] say, the formula is false when P is false and R is true, or Q is false and R is true.

3.5.3 Finding normal forms using truth tables

We have already seen that truth tables are very versatile. Here we consider how they can be used to compute disjunctive and conjunctive normal forms of propositional formulas.

A second method for computing a disjunctive normal form is the following. Let A be a formula with propositional variables $P_1, ..., P_n$.

Step 1: Compute the truth table for A.

Step 2: For each row i in the truth table, let

$$B_i = \begin{cases} \bar{P}_1 \wedge ... \wedge \bar{P}_n & \text{if the entry for } A \text{ is } 1 \\ \bot & \text{otherwise} \end{cases}$$

where

$$\bar{P}_i = \begin{cases} P_i & \text{if } v_i P_i = 1 \\ \neg P_i & \text{otherwise} \end{cases}$$

and v_i is the valuation of row i. B_i is sometimes called the 'correspondent' (or 'minterm') of row i.
Then, A has the disjunctive normal form

$$B_1 \vee ... \vee Bk.$$

In fact, this also gives us a method to find a propositional formula in disjunctive normal form for any truth table.

We give an example. Assume we want to find a propositional formula A with the Boolean interpretation as given by the following truth table.

P	Q	A
0	0	1
0	1	0
1	0	1
1	1	0

Think of 0 representing 'switched off' and 1 representing 'switched on'. The 'switched off' parts of the output are in the second and fourth line. For these we just use $\bot$. The 'switched on' parts of the output are in the first and third line.

[15] More or less immediately; when one reads this for the first time a little reflection is needed to see why.

For the first line the inputs are 0 for both variables, so we have to pick$\neg P_1 \wedge \neg P_2$ for that line. For the third the inputs are 1 and 0 respectively, so we want to pick $P_1 \wedge \neg P_2$ for that line, giving altogether

$$\bot \vee (P_1 \wedge \neg P_2) \vee \bot \vee (\neg P_1 \wedge \neg P_2)$$

and after simplification:

$$(P_1 \wedge \neg P_2) \vee (\neg P_1 \wedge \neg P_2).$$

This is a propositional formula in disjunctive normal form.

Why does this method to compute the disjunctive normal form of a formula work? To give some hints note that a disjunction is true, if at least one of the disjuncts is true. Thus, the disjunction $B_1 \vee ... \vee B_k$ is true when one of the B_i that are not false is true. For example, B_3 corresponding to the third row in the example is true if P_1 and $\neg P_2$ are both true. Thus, $B_3 = P_1 \wedge \neg P_2$.

We leave it to you as an exercise to adapt the method to compute conjunctive normal forms from truth tables. Basically, we can use the two step algorithm above (for extracting a DNF from the given truth table), but switch the roles of 1 and 0, and $\vee$ and $\wedge$.

Although these methods provide easy ways for computing disjunctive and conjunctive normal forms, the complete truth table always needs to be constructed first—this requires exponential time and is not practical unless the formulas involved are small.

Notice also that the normal forms computed by the two methods may not be the same, but are equivalent. The formulas obtained may also not be the 'simplest' (e.g., contain as few symbols as possible). Finding 'simple' normal forms is a challenging problem, which we do not discuss in this course, but this is an important question in hardware design and many AI applications.

We illustrate these observations by reconsidering the last example. Above we computed the following as the disjunctive formal form for a formula A defined by the truth table given above.

$$(P_1 \wedge \neg P_2) \vee (\neg P_1 \wedge \neg P_2)$$

If we convert it to conjunctive normal form (which means we only have to apply Step 4 of the CNF normalisation algorithm above) we get

$$(P_1 \vee \neg P_1) \wedge (P_1 \vee \neg P_2) \wedge (\neg P_2 \vee \neg P_1) \wedge (\neg P_2 \vee \neg P_2).$$

We carry out some simplification steps:

$$\begin{aligned}
&(P_1 \vee \neg P_1) \wedge (P_1 \vee \neg P_2) \wedge (\neg P_2 \vee \neg P_1) \wedge (\neg P_2 \vee \neg P_2) && \\
\equiv &\top \wedge (P_1 \vee \neg P_2) \wedge (\neg P_2 \vee \neg P_1) \wedge (\neg P_2 \vee \neg P_2) && \text{using } A \vee \neg A \equiv \top \\
\equiv &(P_1 \vee \neg P_2) \wedge (\neg P_2 \vee \neg P_1) \wedge (\neg P_2 \vee \neg P_2) && \text{using } \top \wedge A \equiv A \\
\equiv &(P_1 \vee \neg P_2) \wedge \neg P_2 && \text{by comm., absorption} \\
\equiv &\neg P_2 && \text{by } 2\times \text{ comm., absorption}
\end{aligned}$$

And indeed, if we check the original table then we see that the output is 1, if and only if the second input for P_2 is 0. So given a truth table it is always worth checking whether there is a simple propositional formula to read off.

3.5.4 Boolean functions

Recall, $\mathbb{B} = \{0, 1\}$ is the (Boolean) truth value set. We briefly turn to the question which functions from (some power of) $\mathbb{B}$ to $\mathbb{B}$ can be seen as the interpretations of connectives, but also as interpretations of propositional formulas.

Definition 14

A Boolean function f is a function from $\mathbb{B}^n$ to $\mathbb{B}$.

In the Boolean semantics, each connective of propositional logic can be viewed as being interpreted by a Boolean function. For example, negation $\neg$ can be viewed as the function $f_\neg : B \to B$ defined by the function table on the left. This defines that 1 is mapped to 0 and 0 to 1.

x	$f_\neg$
0	1
1	0

x_1	x_2	$f_\wedge$
0	0	0
0	1	0
1	0	0
1	1	1

Conjunction $\wedge$ can be viewed as the function $f_\wedge : B^2 \to B$ defined by the function table on the right. That is, $f_\wedge$ maps $(1, 1)$ to 1 and all other tuples to 0.[16] Note the correspondence between these function tables are the standard truth tables for $\neg$ and $\wedge$.

In a similar way, all the other connectives we considered can be viewed as Boolean functions. Namely, $f_\vee$, $f_\to$ and $f_\leftrightarrow$ are also functions from $\{0, 1\} \times \{0, 1\}$ to $\{0, 1\}$.

Their function tables can be obtained in the expected way from the standard truth tables of the connectives.

Theorem 3.8

The number of possible n-place Boolean functions is 2^{2^n} .

Assume the list of propositional variables is $P_1, P_2, ..., P_n$. Then there are 2^n possible valuations over these variables. We may think of tuples of these as the elements of $\mathbb{B}^n$. For example, $(0, 0, 1, 0, 1)$ is the valuation which makes the following assignment to the variables $P_1, ..., P5$.

P_1	P_2	P_3	P_4	P_5
0	0	1	0	1

Since the target of a Boolean function has two elements (there are two values a tuple can be mapped to), there are 2^{2^n} many ways to define a Boolean function from $\mathbb{B}^n$ to $\mathbb{B}$

We can view each Boolean function as defining a different logical connective (some useful and widely used, some less useful). Let us consider the possible one-place Boolean functions—they are:

x	f_1	f_2	f_3	f_4
0	0	0	1	1
1	0	1	0	1

We note that, $f_1(x) = 0$ for each $x \in \mathbb{B}$. This means f_1 represents $\bot$. In a similar way, f_1 represents $\bot$. f_3 represents negation. f_2 is not interesting as a connective, because f_2 is the identity function on $\mathbb{B}$ (we have that $f_2(x) = x$, for each $x \in \mathbb{B}$).

The possible two-place Boolean functions are:

[16] In general, any function with any number of arguments can be defined by a function table (also called operation table). In the case of binary functions, function tables are often presented in this form

x_1	x_2	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8
0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

x_1	x_2	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}	f_{16}
0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

We have, for each $(x_1, x_2)) \in \mathbb{B}^2$:

$f1(x_1, x_2) = 0$ so this is represents by $\top$.

$f_{16}(x_1, x_2) = 1$ so this is represents by $\bot$

$f_4(x_1, x_2) = x_1$ so this represents a propositional variable

$f_6(x_1, x_2) = x2$ so this also represents a propositional variable

Similarly, f_2 represents $\wedge$ and f_8 represents $\vee$ and so on. We have not defined as many binary connectives as Boolean functions over $\mathbb{B}$, but what you should be able to work out for the two-dimensional case is that every Boolean function can be expressed as a propositional formula using x_1 and x_2 as propositional variables. Remarkably, this generalises:

Theorem 3.9

Every Boolean function $f : \mathbb{B}^n \to \mathbb{B}$ can be expressed as a propositional formula using $x_1, ..., x_n$ as propositional variables.

To see this, simply use the truth table method for computing normal forms.

There are a lot more interesting things that can be said about Boolean functions. During your studies, you will notice them being used in several areas of computer science. In logic, Boolean functions are useful for answering questions relating to the adequacy of sets of connectives. A set of logical connectives is adequate, if all other connectives are expressible in terms of these. For example, $\neg$, $\wedge$ and $\vee$ are adequate, in fact, $\neg$ and$\wedge$ alone, or $\neg$ and $\vee$ alone are already adequate, but just one of these connectives is not adequate. Adequacy of connectives is related to the functional completeness of logical gates in the design of logical switching circuits, where it is desirable to limit the number of different types of logic gates used and their complexity. Such questions are discussed in COMP12111 Fundamentals of Computer Engineering.

When programming

As already pointed out, many programming languages allow the user to build propositional expressions. By replacement of equivalents, for example, two expressions that arise from each other by applying the rule for double negation or De Morgan‘s law, will always evaluate to the same value. This means (as we already observed) that you can use these laws to rewrite the propositional expressions in your code to turn them into something that is more easily understandable (for example by putting them into conjunctive normal form or disjunctive normal form). This makes your code easier to read for others, but it also becomes easier to check whether what has been implemented is as intended. We know that implication can be equivalently expressed by using the other connectives and this is the reason why many programming languages do not have an implication connective.

Exercises

Exercise 22. Use the distributivity law to show that
$(A \wedge B) \vee (C \wedge D) \equiv (A \vee C) \wedge (A \vee D) \wedge (B \vee C) \wedge (B \vee D)$.
Give the appropriate equivalence for the case where there are three propositional formulas in each bracket.

Exercise 23. In Step 4 of the CNF algorithm a disjunction of n conjunctions is transformed into a conjunction of disjunctions. Work out how many disjunctions does the obtained conjunction contain? Explain your answer.

Exercise 24. Show that for arbitrary propositional formulas A and B we have that the Boolean interpretation of$A \wedge (A \vee B)$, and that of$A \vee (A \wedge B)$, is the same as that of A.

CExercise 25. (On Sheet 6) Consider the following propositional formulas.

(i) $((P \wedge \neg Q) \rightarrow (R \wedge \neg P)$

(ii) $\neg(P \wedge Q \wedge \neg R) \vee (\neg R \wedge P)$

(iii) $\neg(P \vee ((Q \vee P) \wedge \neg(Q \wedge P)))$

(iv) $(\neg P \vee \neg Q) \wedge (\neg P \vee (Q \rightarrow R))$

In each case
(a) Apply the CNF algorithm to transform the formula into conjunctive normal form. Then simplify the obtained formula as far as you can.
(b) Apply the DNF algorithm to transform the formula into disjunctive normal form. Then simplify the obtained formula as far as you can.

CExercise 26. (On Sheet 6) For the following propositional formulas, establish whether they are tautologies, without using truth tables or Natural Deduction, by using the fundamental laws we considered. Hint: one possibility is to find a conjunctive normal form first, and then simplify.

(i) $(P \wedge \neg Q) \vee Q \vee \neg(P \rightarrow Q)$,

(ii) $(P \wedge Q) \rightarrow ((P \wedge \neg Q) \vee (P \wedge Q))$,

(iii) $((P \wedge Q) \rightarrow R) \rightarrow (P \rightarrow (Q \rightarrow R))$,

(iv) $((P \rightarrow Q) \wedge (Q \rightarrow R)) \rightarrow (P \rightarrow R)$,

(v) $\neg(P \wedge Q) \rightarrow (\neg P \vee \neg Q)$.

Exercise 27. For the following propositional formulas give a DNF. Then simplify them as far as you can.

(i) $(P \vee \neg Q) \wedge \neg(P \to \neg R)$,
(ii) $\neg(P \to Q) \wedge (P \vee Q \vee \neg R)$,
(iii) $\neg(P \to Q) \to (P \wedge R \wedge \neg Q)$,
(iv) $(P \vee \neg Q) \wedge \neg(R \vee \neg P)$.

Exercise 28. Determine which of the following propositional formulas are semantically equivalent.

(i) $(P \to (Q \vee R)) \wedge (\neg Q \to (P \vee R))$ (ii) $\neg(\neg Q \wedge \neg R)$
(iii)$\neg Q \to (S \to R)$

EExercise 29. (On Sheet 6) For each of the following functions give a propositional formula whose interpretation is the given function. Then give a conjunctive normal form for your propositional formula and simplify the result as far as you can.

(i) For the Boolean function $f : \mathbb{B}^3 \to \mathbb{B}$ given by the following table.

x_1	x_2	x_3	f
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(ii) For the Boolean function $f : \mathbb{B}^3 \to \mathbb{B}$ given by the following table.

x_1	x_2	x_3	f
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Exercise 30. Consider this propositional formula.

$P \to ((Q \vee R) \to (R \to \neg P))$(i) Use the truth table method to give a disjunctive normal form for the formula.

(ii) Use the DNF algorithm to transform the formula into disjunctive normal form, simplifying where you can using the fundamental laws we have considered.

(iii) (Optional) The results obtained are equivalent (why?) but not the same. Try to describe the general difference in the disjunctive normal forms obtained by the two methods.

Exercise 31.
(i) Extract the conjunctive normal form from the truth tables of the following propositional formulas.

(a) $P \vee \neg Q \vee \neg R$

(b) $(P \wedge \neg Q) \rightarrow (R \wedge \neg P)$

(ii) Describe a general method to compute conjunctive normal forms from the truth table of a propositional formula.

Exercise 32. In this section we gave the definition of $\neg$ and $\wedge$ as a function in the Boolean algebra $\mathbb{B}$.

Define the functions over B that define the interpretation of the other connectives of propositional logic. Include their function tables in your answer.

3.6 First-order logic

Propositional logic formalises the key phrases and, or, implies, iff and not, but two of the key phrases from Section 2.2 (of Andrea's lecture notes) do not occur, namely 'for all' and 'there exists'. The reason for this is that these are more complicated, and require more 'infrastructure' in our logic. Even in Section 2.2 it was noted that we have to specify where the two phrases have to hold, and this is something we have to worry about in a formal logic system as well.

In this section we introduce the language of first-order logic, which is arguably the most important logical formalism in computer science and mathematics. Almost anything that we may want to express in mathematics and computer science can be expressed in first-order logic. We mostly refer to first-order formulas simply as formulas.

First-order logic is an extension of propositional logic and most of what we learned for propositional logic and propositional formulas, holds also for first-order logic and formulas of first-order logic. However, since first-order logic is more complex, we don't study it in the same formal detail as propositional logic. Instead, we focus on translating English sentences into the formalism, and on the use of equivalences to simplify formulas, without looking into how these equivalences are formally proved.

3.6.1 The language of first-order logic

In order to allow the formation of formulas that might be interpreted by sentences such as 'for all $i \in \mathbb{N}$ there exists $j \in \mathbb{N}$ such that $j \leq i$' we need to include at least the following building blocks:

- Variables (other than propositional variables) that are placeholders for 'things', for example numbers, to which we may apply 'for all' and 'there exists'.
- Predicate symbols which allow us to talk about properties, such as the primeness of numbers,

  $$Prime(2)$$

  or relations, such as the $<$ relation on numbers, the $=$ relation on numbers, or the betweenness relation, that we may expect to hold for some terms.

  $$2 < 3 \qquad a = 3 \qquad Between(a, b, c)$$

- Possibly symbols that denote functions such as multiplication of numbers or taking the additive inverse of a number

  $$m(1, 5) \qquad m(x, y) \qquad -(1)$$

  .

Because there are so many more building blocks than in the language of propositional formulas, the rules for stating how formulas may be built become more complicated. We here give an outline of first-order logic, but we study it in less detail.

Definition 15

In our definition of the language of first-order logic we assume that apart from propositional variables, we also have a supply of

- constants denoted $a, b, c, ...$ or $a_1, a_2, ...,$
- variables denoted $x, y, z, ...$ or $x_1, x_2, ...,$
- function symbols denoted $f, g, h, ...$ or $f_1, f_2, ...,$ each of which takes a known, fixed number of arguments, and
- predicate symbols denoted $P, Q, R, ...$ or $P_1, P_2, ...,$ each of which takes a known fixed number of arguments.

The constants, function symbols and predicate symbols are domain-specific symbols, formally called **non-logical symbols** or parameters, of our language as their interpretation is not fixed (unlike that of the variables and connectives) but depend on our domain of application and what information we would like to encode as first-order formulas.

We may regard constants as nullary (i.e., zero-place) function symbols, and propositional variables as nullary (i.e., zero-place) predicate symbols.
We have to introduce the notion of a term, which is defined in the following way:

Definition 16: First-order term

- If a is a constant then a is a term.
- If x is a variable then x is a term.
- If $t_1, t_2, ..., t_n$ are terms, and f is a function symbol that takes n arguments, then $f(t_1, t_2, ..., t_n)$ is a term.

Formulas of first-order logic are then defined by the following:

Definition 17: First-order formula

- If P is a propositional variable then P is a formula.
- If Q is a predicate symbol that takes n arguments and $t_1, t_2, ...t_n$ are terms then $Q(t_1, t_2, ...t_n)$ is a formula.
- $\top$ and$\bot$ are formulas.
- If A is a formula then $\neg A$ is a formula.
- If A and B are formulas then $A \wedge B$, $A \vee B$,$A \rightarrow B$ and $A \leftrightarrow B$ are formulas.
- If A is a formula and x is a variable then$\forall x.A$ and $\exists x.A$ are formulas.

This means the logical connectives of the language of first-order logic are those of propositional logic, namely $\top, \bot, \neg, \wedge, \vee, \rightarrow$ and $\leftrightarrow$, plus $\forall x$ and $\exists x$ for each variable x in the language. $\forall x$ is a universal quantifier, and $\exists x$ is an existential quantifier.

In order not to have to write too many brackets, we assume the following binding precedence for the extended set of connectives (from highest to lowest).

$$\neg \qquad \forall x \qquad \exists x$$

$$\vee \qquad \wedge$$

$$\rightarrow \qquad \leftrightarrow$$

This means the binding precedence is as before except that $\forall x$ and $\exists x$ have the same precedence as $\neg$.

3.6.2 Translating English sentences to first-order logic

To determine whether a propositional formula is true, we need a valuation which tells us something about each propositional variable. In first-order logic we have to specify the following things:

- A non-empty set X to represent the set of possible values the variables can take on, called the **domain of interpretation** (for example the natural numbers),
- For each function symbol which takes n arguments, a function from X^n to X (for example we may interpret a binary function symbol m by the multiplication function from $\mathbb{N}$ to $\mathbb{N}$),
- For each predicate symbol which takes n arguments, a set of n-tuples (elements of X^n) which represent the lists of objects which have the relationship specified (for example we my interpret a unary predicate symbol $Prime$ by the set of prime numbers, or a binary predicate symbol $GreaterThan$ by the set $\{(x, y) \in \mathbb{N} \mid x > y\}$).

Then, given a first-order formula, we can work out whether it is true for this interpretation by evaluating it using rules similar to the ones we gave for the propositional connectives. The main new idea is that in a formula of the form $\forall x.F$ there may be occurrences of the variable x in F, and it is true relative to an interpretation if when we use the specifications above we find that F is true when we replace the x by every element of x. Similarly $\exists x.F$ is true if we can make F true by replacing occurrences of x in F with some elements of X. When we carry out this replacement, we can interpret terms by using the specified function to interpret each function symbol. If we replace all variables in the term by values from X, then we can evaluate these functions and so the whole terms is evaluated to an element of X. To work out is a predicate symbol is true, we first evaluate all the terms is is applied to to obtain a tuple of elements of X. If this tuple is in the set we specified for that predicate symbol, we output 1, and otherwise 0. Now the other connectives can be evaluated in the same way as in propositional logic. This is not a rigorous description of the process, however as indicated above spelling out all the details and corner cases for first-order formulas is quite intricate so we do not do it formally in this unit.

Often it is necessary to find a formula that expresses a particular property or statement for a domain of interpretation. We give one example here in the form of primeness, Definition 17 in Section 2.3 (Andrea's part). We repeat the definition here (and limit ourselves to natural numbers).

Definition: Def. 17 from Andrea's part

An element $n \neq 1$ of $\mathbb{N}$ is prime if and only if for all elements $k4$ and l of $\mathbb{N}$ it is the case that

n divides $k \cdot l$

implies

n divides k

or

n divides l.

This defines when a natural number is prime. Our aim is to express this definition as a first-order formula.

First, we need to pick the non-logical symbols to use. Let us use the unary predicate symbol P for the property of natural numbers being prime. The definition also requires a binary predicate symbol, we choose to use D, for the divides relation between two numbers. We also pick a binary function symbol m for the multiplication operation on two numbers. Finally, we need to exclude the number 1 from the definition. Let us therefore use the constant u for the number 1, and E for the equality relation.

We let the domain of interpretation be the set of natural numbers $\mathbb{N}$, and interpret the non-logical symbols we have chosen as follows.

- u is interpreted as 1,
- m is interpreted as the multiplication operation on the natural numbers, and for any terms t and t':
- $P(t)$ is interpreted as 1 if and only if t is prime, i.e., belongs to the set of prime numbers,
- $D(t, t')$ is interpreted as 1 if and only it t divides t',
- $E(t, t')$ is interpreted as 1 if and only if $t = t'$, i.e., t and t' are the same number.

The right-hand side of the 'if and only if' statement of the above definition of primeness is expressed by the following formula:

$$\forall y.\forall z.D(x, m(y, z)) \rightarrow (D(x, y) \vee D(x, z)).$$

The full definition of primeness, including the exclusion of 1 from the definition, is captured by:

$$\forall x.(\neg E(x, u) \rightarrow P(x) \leftrightarrow (\forall y.\forall z.D(x, m(y, z)) \rightarrow (D(x, y) \vee D(x, z)))).$$

Note that almost the same formula can be assumed to talk about primeness for the integers merely by changing the domain of interpretation to Z and changing the precondition (see Definition 17 in Section 2.3)!

Every computer scientist has to be able to do this kind of modelling and representation of information in first-order logic. As the example illustrates, modelling involves being clear about the domain of interpretation, picking appropriate non-logical symbols to use and writing statements made in English as first-order formulas.

3.6.3 Equivalent formulas and normal forms

Many notions of propositional logic have natural generalisations in first-order logic, and the intuition remains the same. This is the case for the notion of subformula. We define:

Definition 18

If A is a formula and A viewed as a string of symbols occurs as a substring of another formula B, then A is a subformula of B. Note that B is also a subformula of B, for any formula B.

Other notions such as the semantic equivalence of two first-order formulas are more complicated than for propositional formulas. There is though some similarity to the notion of semantic equivalence of propositional formulas in the power set semantics, because the interpretation of first-order formulas is defined relative to a domain of interpretation as well as the two valuation functions and interpretations of the non-logical symbols.

Definition 19

In first-order logic, two formulas A and B are semantically equivalent, denoted by $A \equiv B$, if and only if in all interpretations for all non-empty domains, the interpretation of A is the same as the interpretation of B.

Examples of semantic equivalences in first-order logic are $(A \rightarrow B) \equiv (\neg A \vee B)and\neg\neg A \equiv A$. In fact, all fundamental semantic equivalences in propositional logic are also semantic equivalences in first-order logic.

This is a consequence of this Substitution Theorem generalised from Theorem 3.3 by substituting first-order formulas for propositional variables.

Theorem 3.10: Substitution into propositional equivalences

. Let A and B be any propositional formulas. If

$$A(P_1, ..., P_n) \equiv B(P_1, ..., P_n)$$

and $C_1, ..., C_n$ are first-order formulas, then

$$A(C1, ..., Cn) \equiv B(C_1, ..., C_n)$$

.

(The meaning of the notation is the same as in the Substitution Theorem for propositional logic)

This means lots of semantic equivalences are easily obtained by substitution of first-order formulas for propositional variables in propositional equivalences.

Semantic equivalences are useful to equivalently transform and manipulate formulas; this is especially important to first-order logic, since the truth table method is not applicable to first-order formulas (because of the presence of first-order variables and quantifiers). In first-order logic the basis for equivalence preserving transformation is given by this generalisation of the Equivalent Replacement Theorem:

Theorem 3.11: Equivalent Replacement Theorem

. Let A and B be formulas. Suppose $A \equiv B$ and A is a subformula of $C(...A...)$. Then $C(...A...) \equiv C(...B...)$, where $C(...B...)$ is obtained by putting B for one occurrence of A in $C(...A...)$.

Useful fundamental semantic equivalences involving quantifiers are summarised in this theorem.

Theorem 3.12: Fundamental semantical equivalences

.

$\neg\forall x.A \equiv \exists x.\neg A$	duality
$\neg\exists x.A \equiv \forall x.\neg A$	duality
$\forall x.\forall y.A \equiv \forall y.\forall x.A$	
$\exists x.\exists y.A \equiv \exists y.\exists x.A$	
$\forall x.(A \wedge B) \equiv (\forall x.A) \wedge (\forall x.B)$	
$\exists x.(A \vee B) \equiv (\exists x.A) \vee (\exists x.B)$	

The first two equivalences are important because they are useful to pull negation inside quantifiers, which is how they are often used. They also tell us tha t$\forall$ and $\exists$ are dual operators. They imply (by using equivalent replacement) that:

$$\neg\forall x.\neg A \equiv \exists x.A$$

$$\neg\exists x.\neg A \equiv \forall x.A$$

So one quantifier can be defined in terms of the other quantifier and negation.

The next two equivalences in the theorem say that the order in a sequence of universal quantifiers (or existential quantifiers) does not matter. But note that

$$\forall x.\exists y.A \not\equiv \exists y.\forall x.A$$

$$\exists x.\forall y.A \not\equiv \forall y.\exists x.A,$$

which means universal and existential quantifiers cannot be switched.

The last two equivalences in the theorem say that universal quantifiers distribute over conjunction, and existential quantifiers distribute over disjunction. However, note that

$$\forall x.(A \vee B) \not\equiv (\forall x.A) \vee (\forall x.B)$$

$$\exists x.(A \wedge B) \not\equiv (\exists x.A) \wedge (\exists x.B)$$

.

There are more fundamental semantic equivalences involving quantifiers, but we limit ourselves to the ones in the Theorem above, as no variable binding issues arise, which we do not discuss in this course.

There are the notions of conjunctive normal form and disjunctive normal form for first-order logic, but because various extra notions and notations are needed, and because of variable binding issues, this requires more time to describe.

However, the above fundamental laws and results are already sufficient to find semantically equivalent formulas that look much simpler and improve the intelligibility of formulas in a lot of cases. For example, using what we have discussed we can perform the following transformation steps.

$$
\begin{aligned}
&\neg\exists x.\forall y.(P(x) \to Q(x,y)) && \\
\equiv &\forall x.\neg\forall y.(P(x) \to Q(x,y)) && \text{duality} \\
\equiv &\forall x.\neg\forall y.(\neg P(x) \vee Q(x,y)) \vee B && A \to B \equiv \neg A\text{, Subst. and Equiv. Repl} \\
\equiv &\forall x.\exists y.\neg(\neg P(x) \vee Q(x,y)) && \text{duality} \\
\equiv &\forall x.\exists y.(\neg\neg P(x) \wedge \neg Q(x,y)) && \text{De Morgan } \vee\text{, Subst. and Equiv. Repl.} \\
\equiv &\forall x.\exists y.(P(x) \wedge \neg Q(x,y)) && \neg\neg A \equiv A\text{, Subst. and Equiv. Repl.}
\end{aligned}
$$

Again performing these steps results in a formula that is simpler and easier to understand.

Equivalence preserving transformations based on fundamental equivalences and equivalent replacement, are useful for showing equivalence of two formulas, for simplifying formulas, perhaps reducing them to $\top$ or $\bot$, and transforming formulas into conjunction or disjunctive normal forms.

Exercises

Exercise 33. Consider a first-order language with the two unary predicate symbols H and O. As the domain of interpretation we use the set of current undergraduate students in the School of Computer Science. And, we use the following interpretations:

- H(x): x has taken and passed the health-and-safety test.
- O(x): x has an out-of-hours pass for the building.

What is the interpretation of the following first-order formulas in this domain? Explain your answers.
(i) $\forall x.(O(x) \to H(x))$
(ii) $\forall x.(H(x) \to O(x))$

CExercise 34. (On Sheet 6) Consider a first-order language with

- a binary predicate symbol T , a binary predicate symbol L, and no function symbols.

Take as the domain of interpretation the set of all first year students in the Department. Assume that

- T(x, y) means that x and y are in the same tutorial group, and
- L(x, y) means that x and y are in the same lab group.

Consider the following formulas.

(i) $\forall x.\forall y.(T(x,y) \to L(x,y))$

(ii) $\forall x.\forall y.(\neg T(x,y) \to \neg L(x,y))$

(iii) $\forall x.\exists y.(T(x,y) \wedge L(x,y))$

(iv) $\exists x.\exists y.(L(x,y) \wedge \neg T(x,y))$

(v) $\forall x.\exists y.(\neg L(x,y) \vee \neg T(x,y))$

(vi) $\exists x.\forall y.\neg T(x,y)$

Say in each case what is the interpretation of the formula in the given domain? Explain your answers as best you can using your knowledge of how the tutorial and lab groups are organised. Hint: The tutorial groups with the same letter W, X, Y or Z form a lab group. For some course units students in tutorial groups with the letter M join students with W, so the lab groups are W+M, X, Y and Z.

Exercise 35. Express each formula in the previous exercise as best as possible in idiomatic English without using variables x and y. Assume the domain of interpretation, and the interpretation of T and L, are as in that exercise. In your answers feel free to use ‘student‘ as a shortcut for ‘first year student in the Department‘.

CExercise 36. (On Sheet 6) For the language and interpretation defined in Exercise 34, give formulas which express the following. We use ‘student‘ here as a shortcut for ‘first year student in the School‘.

(i) Every student is in the same tutorial group as him-or herself.

(ii) For every student there is a student who is in a different tutorial group.

(iii) There is a student who is in the same labgroup as all students.

(iv) No student is in the same labgroup as all the other students.

(v) No two students who are in the same tutorial group are not in the same lab group.

EExercise 37. (On Sheet 6) Model the following definitions from Section 2.3 (of Andrea‘s lecture notes) in first-order logic and translate them into first-order formulas. Make sure you state what your constants, function symbols and predicate symbols are and what your domain of interpretation is as well as how each symbol should be interpreted.

(i) Associativity of an operation, Definition 18.

(ii) Existence of an inverse for every element with respect to an operation, Definition 21.

(iii) Injectivity of a function with the same source and target, Definition 22.

(iv) Surjectivity of a function with the same source and target, Definition 23.

(v) Bijectivity of a function with the same source and target, Definition 24.

(vi) One function with the same source and target being the inverse of another, Definition 25.

You may want to think about why we demand that our functions have the same source and target.

CExercise 38. (On Sheet 6) Simplify the following formulas as far as you can by using steps we have used to transform propositional formulas to conjunctive normal form, fundamental semantic equivalences involving quantifiers and the results stated in this section about the preservation of semantic equivalence for first-order formulas.

(i) $\neg\forall x.(P(x) \to \exists y.Q(x,y))$

(ii) $\exists x.\neg\exists y.(Q(y) \wedge R(x)) \to \forall z.P(z,x,y)$

(iii) $\forall y.\neg\exists z.(P(y,z) \to \neg Q(z))$

(iv) $\exists z.\forall y.(\neg P(y) \wedge Q(z)) \to R(y,z)$

COMP11120, Semester 1
Assessed Exercise Sheet 4

Core exercises

- For Exercise 1: do any two parts from (ii) to (vi)
- For Exercise 4: do any two parts
- For Exercise 5: do one of parts (i)-(ii) and one of parts (iii)-(iv)

Extensional exercises

- For Exercise 3 do the case where $\star$ is $\rightarrow$, and one of $\star \in \{\wedge, \vee\}$
- Complete Exercise 6

COMP11120, Semester 1
Assessed Exercise Sheet 5

Core exercises

- For Exercise 7 do any three parts
- For Exercise 10 and Exercise 11, do all of Exercise 10, and one part from (i)-(iii), part (iv), and one part from (vi)-(viii) from Exercise 11
- Complete Exercise 17

Extensional exercises

- Complete Exercise 8
- Complete Exercise 13

COMP11120, Semester 1
Assessed Exercise Sheet 6

Core exercises

- From Exercise 25 and Exercise 26: do one of (i) and, (iv) and one of (ii) and (iii) from Exercise 25, and any two parts of Exercise 26
- For Exercise 34 and Exercise 36: do any three parts of Exercise 34 and any three parts of Exercise 36
- Complete Exercise 38

Extensional exercises

- Complete Exercise 29
- Complete Exercise 37